

How Good Is Multi-Pivot Quicksort?

MARTIN AUMÜLLER, MARTIN DIETZFELBINGER, and PASCAL KLAUE,
Technische Universität Ilmenau

Multi-Pivot Quicksort refers to variants of classical quicksort where in the partitioning step k pivots are used to split the input into $k + 1$ segments. For many years, multi-pivot quicksort was regarded as impractical, but in 2009 a two-pivot approach by Yaroslavskiy, Bentley, and Bloch was chosen as the standard sorting algorithm in Sun's Java 7. In 2014 at ALENEX, Kushagra et al. introduced an even faster algorithm that uses three pivots. This article studies what possible advantages multi-pivot quicksort might offer in general. The contributions are as follows: Natural comparison-optimal algorithms for multi-pivot quicksort are devised and analyzed. The analysis shows that the benefits of using multiple pivots with respect to the average comparison count are marginal and these strategies are inferior to simpler strategies such as the well-known median-of- k approach. A substantial part of the partitioning cost is caused by rearranging elements. A rigorous analysis of an algorithm for rearranging elements in the partitioning step is carried out, observing mainly how often array cells are accessed during partitioning. The algorithm behaves best if three to five pivots are used. Experiments show that this translates into good cache behavior and is closest to predicting observed running times of multi-pivot quicksort algorithms. Finally, it is studied how choosing pivots from a sample affects sorting cost. The study is theoretical in the sense that although the findings motivate design recommendations for multipivot quicksort algorithms that lead to running-time improvements over known algorithms in an experimental setting, these improvements are small.

CCS Concepts: • **Theory of computation** → **Sorting and searching**; *Divide and conquer*;

Additional Key Words and Phrases: Sorting, quicksort, multi-pivot

ACM Reference Format:

Martin Aumüller, Martin Dietzfelbinger, and Pascal Klaue. 2016. How good is multi-pivot quicksort? ACM Trans. Algorithms 13, 1, Article 8 (October 2016), 47 pages.

DOI: <http://dx.doi.org/10.1145/2963102>

1. INTRODUCTION

Quicksort [Hoare 1962] is an efficient standard sorting algorithm with implementations in practically all algorithm libraries. Following the divide-and-conquer paradigm, on an input consisting of n elements quicksort uses a pivot element to partition its input elements into two parts, the elements in one part being smaller than or equal to the pivot, the elements in the other part being larger than or equal to the pivot, and then uses recursion to sort these parts.

In k -pivot quicksort, k elements of the input are picked and sorted to get the pivots $p_1 \leq \dots \leq p_k$. Then the task is to partition the remaining input according to the $k + 1$

Work was carried out while the first author was affiliated with Technische Universität Ilmenau. Part of the work was done while the third author was a Master's student at Technische Universität Ilmenau.

Authors' addresses: M. Aumüller, IT University of Copenhagen, Rued Langgaards Vej 7, 2300 København S, Denmark; email: maau@itu.dk; M. Dietzfelbinger, Fakultät für Informatik und Automatisierung, Technische Universität Ilmenau, 98683 Ilmenau, Germany; email: martin.dietzfelbinger@tu-ilmenau.de; P. Klaue, 3DInteractive GmbH, 98693 Ilmenau, Germany; email: pklaue@3dinteractive.de.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701 USA, fax +1 (212) 869-0481, or permissions@acm.org.

© 2016 ACM 1549-6325/2016/10-ART8 \$15.00

DOI: <http://dx.doi.org/10.1145/2963102>

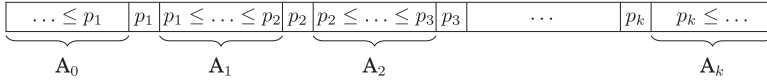


Fig. 1. Result of the partition step in k -pivot quicksort using pivots $p_1, \dots, p_k$.

segments or *groups* defined by the pivots. Segment i , denoted by A_i , consists of elements that are at least as large as p_i and at most as large as p_{i+1} , for $1 \leq i \leq k-1$, and groups A_0 and A_k consist of elements at most as large as p_1 and at least as large as p_k , respectively, see Figure 1. These segments are then sorted recursively. As we will explore in this article, using more than one pivot allows us to choose from a variety of different partitioning strategies. This article will provide the theoretical foundations to analyze these methods.

1.1. History and Related Work

Variants of classical quicksort were the topic of extensive research, such as sampling variants [Sedgewick 1975; Martínez and Roura 2001], variants for equal keys [Sedgewick 1977], or variants for sorting strings [Bentley and Sedgewick 1997]. On the other hand, until 2009 very little work had been done on quicksort variants that use more than one but still a small number of pivots. This is because such variants of quicksort were judged impractical in two independent Ph.D. theses: Sedgewick [1975] had proposed and analyzed a dual-pivot approach that was inferior to classical quicksort in terms of the average swap count. Later, Hennequin [1991] studied the general approach of using $k \geq 1$ pivot elements. According to Wild and Nebel [2012], he found only slight improvements with respect to the average comparison count that would not compensate for the more involved partitioning procedure. A particularly popular variant of multi-pivot quicksort using a large number of pivots is samplesort from Frazer and McKellar [1970]. In that article, it was shown that a multi-pivot approach can be used to achieve an average comparison count very close to the lower bound for comparison-based sorting algorithms. Samplesort has found applications in parallel systems and on GPUs [Leischner et al. 2010], and there also exist well-engineered implementations for standard CPUs [Sanders and Winkel 2004].

In this article, we focus on variants with a small constant number of pivots. While these long had been judged impractical, everything changed in 2009 when a two-pivot quicksort algorithm was introduced as the standard sorting algorithm in Sun’s Java 7. We will refer to this algorithm as the Yaroslavskiy, Bentley, and Bloch algorithm (YBB algorithm). In several previous articles, for example, Wild and Nebel [2012], Kushagra et al. [2014], Aumüller and Dietzfelbinger [2015], and Martínez et al. [2015], this algorithm was called “Yaroslavskiy’s algorithm,” which was motivated by the discussion found at Yaroslavskiy [2009]. The authors were informed [Bloch 2015] that the algorithm should be considered joint work by Yaroslavskiy, Bentley, and Bloch. Wild and Nebel (joined by Neiningner in the full version) [2012, 2016] analyzed a variant of the YBB algorithm and showed that it uses $1.9n \ln n + O(n)$ comparisons and $0.6n \ln n + O(n)$ swaps on average to sort a random input if two arbitrary elements are chosen as the pivots. Thus, this two-pivot approach turned out to improve on classical quicksort—which makes $2n \ln n + O(n)$ comparisons and $0.33..n \ln n + O(n)$ swaps on average—w.r.t. the average comparison count. However, the swap count was negatively affected by using two pivots, which had also been observed for another dual-pivot quicksort algorithm in Sedgewick [1975]. Aumüller and Dietzfelbinger [2013, 2015] showed a lower bound of $1.8n \ln n + O(n)$ comparisons on average for two-pivot quicksort algorithms and devised natural two-pivot algorithms that achieved this lower bound. The key to understanding what is going on here is to note that one can improve the comparison

count by deciding in a clever way with which one of the two pivots a new element should be compared first. While optimal algorithms with respect to the average comparison count are simple to implement, they must either count frequencies or need to sample a small part of the input, which renders them not competitive with the YBB algorithm with respect to running time when key comparisons are cheap. Moreover, Aumüller and Dietzfelbinger [2015] proposed a two-pivot algorithm that makes $2n \ln n + O(n)$ comparisons and $0.6n \ln n + O(n)$ swaps on average—no improvement over classical quicksort in both cost measures—but behaves very well in practice. Hence, the running-time improvement of a two-pivot quicksort approach could not be explained conclusively in these works.

Very recently, Kushagra et al. [2014] proposed a novel three-pivot quicksort approach. Their algorithm compares a new element with the middle pivot first and then with one of the two others. While the general idea of this algorithm had been known before (see, for example, Hennequin [1991] and Tan [1993]), they provided a smart way of exchanging elements. Building on the work of LaMarca and Ladner [1999], they showed theoretically that their algorithm is more cache efficient than classical quicksort and the YBB algorithm. They reported on experiments that gave reason to believe that the improvements of multi-pivot quicksort algorithms with respect to running times are due to their better cache behavior. They also reported from experiments with a seven-pivot algorithm, which ran more slowly than their three-pivot algorithm. We will describe how their (theoretical) arguments generalize to quicksort algorithms that use more than three pivots. In connection with the running-time experiments from Section 9, this allows us to make more accurate predictions than Kushagra et al. [2014] about the influence of cache behavior to running time. One result of the present study will be that it is not surprising that their seven-pivot approach is slower, because it has worse cache behavior than three- or five-pivot quicksort algorithms using a specific partitioning strategy.

In actual implementations of quicksort and dual-pivot quicksort, pivots are usually taken from a small sample of elements. For example, the median in a sample of size $2k + 1$ is the standard way to choose the pivot in classical quicksort. Often this sample contains only a few elements, say, three or five. The first theoretical analysis of this strategy is by van Emden [1970]. Martínez and Roura [2001] settled the exact analysis of the leading term of this strategy in 2001. In practice, other pivot sampling strategies were applied successfully as well, such as the “ninth” variant from Bentley and McIlroy [1993]. In the implementation of the YBB algorithm in Sun’s Java 7, the second- and fourth-largest element in a sample of size five are chosen as pivots. The exact analysis of (optimal) sampling strategies for the YBB algorithm is due to Nebel et al. [2016]. Interestingly, for the comparison count of the YBB algorithm, it is not optimal to choose as pivots the tertiles of the sample; indeed, asymmetric choices are superior from a theoretical point of view. Moreover, it is shown there that—in contrast to classical quicksort with the median of $2k + 1$ strategy—it is impossible to achieve the lower bound for comparison-based sorting algorithms using the YBB algorithm. Aumüller and Dietzfelbinger [2015] later showed that this is not an inherent drawback of a dual-pivot quicksort. Other strategies, such as always comparing with the larger pivot first, make it possible to achieve this lower bound. For more than two pivots, Hennequin [1991] was again the first to study how pivot sampling affects the average comparison count when a “most-balanced” comparison tree is used in each classification, see Hennequin [1991, Tableau D.3].

1.2. Contributions

The main contributions of the present article are as follows:

- (i) In the style of Aumüller and Dietzfelbinger [2015], we study how the average comparison count of an arbitrary k -pivot quicksort algorithm can be calculated. Moreover, we show a lower bound for k -pivot quicksort and devise natural algorithms that achieve this lower bound. It will turn out that the partitioning procedures become complicated and the benefits obtained by minimizing the average comparison count are only minor. In brief, optimal k -pivot quicksort cannot improve on simple and well-studied strategies such as classical quicksort using the median-of- k strategy. Compared with the study of two-pivot algorithms in Aumüller and Dietzfelbinger [2015], the results generally carry over to the case of using $k \geq 3$ pivots. However, the analysis becomes more involved, and we were not able to prove tight asymptotic bounds as in the two-pivot case. The interested reader is invited to peruse Aumüller and Dietzfelbinger [2015] to get acquainted with the ideas underlying the general analysis.
- (ii) Leaving key comparisons aside, we study the problem of rearranging the elements to actually partition the input. We devise a natural generalization of the partitioning algorithms used in classical quicksort, the YBB algorithm, and the three-pivot algorithm of Kushagra et al. [2014] to solve this problem. The basic idea is that as in classical quicksort there exist two pointers that scan the array from left to right and right to left, respectively, and the partitioning process stops when the two pointers meet. Misplaced elements are moved with the help of $k - 1$ additional pointers that store starting points of special array segments. We study this algorithm with regard to three cost measures: (a) the average number of scanned elements (which basically counts how often array cells are accessed during partitioning, see Section 7 or Nebel et al. [2016] for a precise definition), (b) the average number of writes into array cells, and (c) the average number of assignments necessary to rearrange the elements. Interestingly, while moving elements around becomes more complicated during partitioning, this algorithm scans fewer array cells than classical quicksort for certain (small) pivot numbers. We will see that three- and five-pivot quicksort algorithms visit the fewest array cells and that this translates directly into good cache behavior and corresponds to differences in running time in practice. In brief, we provide strong evidence that the running time improvements of multi-pivot quicksort are largely due to its better cache behavior (as conjectured by Kushagra et al. [2014]) and that no benefits are to be expected from using more than five pivots.
- (iii) We analyze sampling strategies for multi-pivot quicksort algorithms with respect to comparisons and scanned elements. We will show that for each fixed order in which elements are compared to pivots there exist pivot choices that yield a comparison-optimal multi-pivot quicksort algorithm. When considering scanned elements, there is one optimal pivot choice. Combining comparisons and scanned elements, the analysis provides a candidate for the order in which elements should be compared to pivots that has not been studied in previous attempts like in Hennequin [1991] and Iliopoulos [2014].

Advice to the Practitioner. We stress that this article is a theoretical study regarding the impact of using more than one pivot in the quicksort framework, which sheds light also on the limitations of this approach. Still, we believe that our theoretical findings lead to some interesting design recommendations for multi-pivot quicksort algorithms. These findings are summarized in Section 9 on experimental evaluation, where they are contrasted with tests regarding empirical running times. We demonstrate that for random permutations one obtains sorting algorithms faster than other approaches such as the YBB algorithm [Wild et al. 2015] or the three-pivot algorithm of Kushagra et al. [2014]. While the differences in running time are statistically significant, the

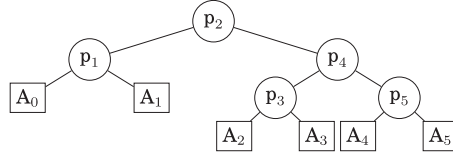


Fig. 2. A comparison tree for five pivots.

relative difference in running time is usually not more than 5%. A lot of care has to be taken in making “library-ready” implementations of these algorithms in terms of their performance on different input types, especially with a focus on handling equal keys in the input. This must remain the objective of future work.

1.3. Outline

In the analysis of quicksort, the analysis of one particular partitioning step with respect to a specific cost measure, for example, the number of comparisons (or assignments or array accesses), makes it possible to precisely analyze the cost over the whole recursion. In Hennequin’s thesis [1991], the connection between partitioning cost and overall cost for quicksort variants with more than one pivot has been analyzed in detail. The result relevant for us is that if k pivots are used and the (average) partitioning cost for n elements is $a \cdot n + O(1)$, for a constant a , then the average cost for sorting n elements is

$$\frac{1}{H_{k+1} - 1} \cdot a \cdot n \ln n + O(n), \quad (1)$$

where H_{k+1} denotes the $(k + 1)$ st harmonic number. In Section 2, we will use the continuous Master theorem from Roura [2001] to prove a more general result for partitioning cost $a \cdot n + O(n^{1-\epsilon})$. Throughout the present article, all that interests us is the constant factor with the leading term. (Of course, for a realistic input size n the lower-order term can have a big influence on the cost measure.)

For the purpose of the analysis, we will consider the input to be a random permutation of the integers $\{1, \dots, n\}$. An element x belongs to group A_i , $0 \leq i \leq k$, if $p_i < x < p_{i+1}$ (see Figure 1), where we set $p_0 = 0$ and $p_{k+1} = n + 1$. When focusing on a specific cost measure, we can often leave aside certain aspects of the partitioning process. For example, in the study of the average comparison count of an arbitrary k -pivot algorithm, we will only focus on *classifying* the elements into their respective groups $A_0, \dots, A_k$ and omit rearranging these elements to produce the actual partition. On the other hand, when focusing on the average swap count (or the average number of assignments necessary to move elements around), we might assume that the input is already classified and that the problem is to rearrange the elements to obtain the actual partition.

In terms of classifying the elements into groups $A_0, \dots, A_k$, the most basic operation is the classification of a single element. This is done by comparing the element against the pivots in some order. This order is best visualized using a comparison tree, which is a binary search tree with $k + 1$ leaves labeled $A_0, \dots, A_k$ from left to right and k inner nodes labeled $p_1, \dots, p_k$ according to inorder traversal. (Such a tree for $k = 5$ is depicted in Figure 2.) Assume a comparison tree λ is given and pivots $p_1, \dots, p_k$ have been chosen. Then a given non-pivot element determines a search path in λ in the usual way; its classification can be read off from the leaf at the end of the path. If the input contains a_h elements of group A_h , for $0 \leq h \leq k$, then the *cost* $\text{cost}^\lambda(a_0, \dots, a_k)$ of a comparison tree λ is the sum over all j of the depth of the leaf labeled A_j multiplied with a_j , that is, the total number of comparisons made when classifying the whole input using λ . A classification algorithm then defines which comparison tree is to

be used for the classification of an element based on the outcome of the previous classifications. The first main result of our article—presented in Section 3—is that in order to (approximately) determine the average comparison count for partitioning given $p_1, \dots, p_k$, we only have to find out how many times on average each comparison tree is used by the algorithm. The average comparison count up to lower-order terms is then the sum over all trees of the product of the average number of times a tree is used and its average cost. Averaging over all pivot choices then gives the average comparison count for the classification. Section 4 applies this result by discussing different classification strategies for three-pivot quicksort.

In Section 5, we will show that there exist two very natural comparison-optimal strategies. The first strategy counts the number of elements $a'_0, \dots, a'_k$ classified to groups $A_0, \dots, A_k$, respectively, after the first i classifications. The comparison tree used in the $(i + 1)$ st classification is then one with minimum cost w.r.t. $(a'_0, \dots, a'_k)$. The second strategy uses an arbitrary comparison tree for the first $n^{3/4}$ classifications and then computes a cost-minimal comparison tree for the group sizes seen in that sample; afterwards, it uses this tree in each of the remaining classifications.

A full analysis of optimal versions of k -pivot quicksort for $k \geq 4$ remains open. In Section 6, we resort to estimates for the cost of partitioning based on experiments to estimate coefficients for average comparison counts for larger k . The results show that the improvements given by comparison-optimal k -pivot quicksort can be achieved in much simpler ways, for example, by combining classical quicksort with the median-of- k pivot sampling technique. Moreover, while choosing an optimal comparison tree for fixed segment sizes is a simple application of dynamic programming, for large k the time needed for the computation renders optimal k -pivot quicksort useless with respect to running time.

Beginning with Section 7, we will follow a different approach, which we hope helps in understanding factors that differ from comparison counts that determine the running time of multi-pivot quicksort algorithms. We restrict ourselves to use a fixed comparison tree for each classification and think only about moving elements around in a swap-efficient or cache-efficient way. At the first glance, it is not clear why more pivots should help. Intuitively, the more segments we have, the more work we have to do to move elements to the segments, because we are much more restrictive about where an element should be placed. However, we save work in the recursive calls, since the denominator in Equation (1) gets larger as we use more pivots. (Intuitively, the depth of the recursion tree decreases.) So, while the partitioning cost increases, the total sorting cost could actually decrease. We will devise an algorithm that builds on the “crossing pointer technique” of classical quicksort. In brief, two pointers move towards each other as in classical quicksort. Misplaced elements are directly moved to some temporary array segment that holds elements of that group. This is done with the help of additional pointers. Moving misplaced elements is in general not done using swaps but rather by moving elements in a cyclic fashion. Our cost measure for this algorithm is the number of *scanned elements*, that is, the sum over all array cells of how many pointers accessed this array cell during partitioning and sorting. For the average number of scanned elements, it turns out that there is an interesting balance between partitioning cost and total sorting cost. In fact, in this cost measure, the average cost drastically decreases from using one pivot to using three pivots, there is almost no difference between three- to five-pivot quicksort, and for larger pivot numbers the average cost increases again. Interestingly, with respect to two other cost measures that look quite similar, we get higher cost as the number of pivots increases.

In Section 8 we turn our attention to the effect of choosing pivots from a (small) sample of elements. Building on the theoretical results regarding comparisons and scanned elements from before, it is rather easy to develop formulae to calculate the

average number of comparisons and the average number of scanned elements when pivots are chosen from a small sample. Example calculations demonstrate that the cost in both measures can be decreased by choosing pivots from a small (fixed-sized) sample. Interestingly, the best pivot choices do not balance subproblem sizes but tend to make the middle groups larger. To get an idea what optimal sampling strategies should look like, we consider the setting that we can choose pivots of a given rank for free, and we are interested in the ranks that minimize the specific cost measure. Our first result in this setting shows that for every fixed comparison tree it is possible to choose the pivots in such a way that on average we need at most $1.4426 \cdot n \ln n + O(n)$ comparisons to sort the input, which is optimal. As a second result, we identify a particular pivot choice that minimizes the average number of scanned elements.

At the end of this article, we report on experiments carried out to find if the theoretical cost measures are correlated to observed running times in practice. To this end, we implemented k -pivot quicksort variants for many different pivot numbers and compared them with respect to their running times. In brief, these experiments will confirm what has been conjectured in Kushagra et al. [2014]: Running times of quicksort algorithms are best predicted using a cost measure related to cache misses in the CPU.

2. SETUP AND GROUNDWORK

We assume that the input is a random permutation $(e_1, \dots, e_n)$ of $\{1, \dots, n\}$. If $n \leq k$, then sort the input directly. For $n > k$, sort the first k elements such that $e_1 < e_2 < \dots < e_k$ and set $p_1 = e_1, \dots, p_k = e_k$. In the *partition step*, the remaining $n - k$ elements are split into $k + 1$ groups $A_0, \dots, A_k$, where an element x belongs to group A_h if $p_h < x < p_{h+1}$. (For the ease of discussion, we set $p_0 = 0$ and $p_{k+1} = n + 1$.) The groups $A_0, \dots, A_k$ are then sorted recursively. We never compare two non-pivot elements against each other. This preserves the randomness in the groups $A_0, \dots, A_k$. In the remainder of this article, we identify group sizes by $a_i := |A_i| = p_{i+1} - p_i - 1$ for $i \in \{0, \dots, k\}$. In the first sections, we focus on analyzing the average comparison count. Let $k \geq 1$ be fixed. Let C_n denote the random variable that counts the comparisons being made when sorting an input of length n , and let P_n be the random variable that counts the comparisons made in the first partitioning step. The average comparison count of k -pivot quicksort clearly obeys the following recurrence, for $n \geq k$:

$$E(C_n) = E(P_n) + \frac{1}{\binom{n}{k}} \sum_{a_0 + \dots + a_k = n - k} (E(C_{a_0}) + \dots + E(C_{a_k})).$$

If $n < k$, we assume cost 0. Now, for $0 \leq \ell \leq n - k$ we collect terms with a common factor $E(C_\ell)$. To this end, fix $j \in \{0, \dots, k\}$ and $\ell \in \{0, \dots, n - k\}$ and assume that $a_j = \ell$. There are exactly $\binom{n - \ell - 1}{k - 1}$ ways to choose the other segment sizes $a_i, i \neq j$, such that $a_0 + \dots + a_k = n - k$. (Note the equivalence between segment sizes and binary strings of length $n - \ell - 1$ with exactly $k - 1$ ones.) Thus, we conclude that

$$E(C_n) = E(P_n) + \frac{k + 1}{\binom{n}{k}} \sum_{\ell=0}^{n-k} \binom{n - \ell - 1}{k - 1} E(C_\ell), \quad (2)$$

which was also observed in Iliopoulos [2014]. (This generalizes the well-known formula $E(C_n) = n - 1 + (2/n) \cdot \sum_{0 \leq \ell \leq n-1} E(C_\ell)$ for classical quicksort and the formulas for $k = 2$ from, for example, Aumüller and Dietzfelbinger [2015] and Wild and Nebel [2012] and $k = 3$ from Kushagra et al. [2014].) For partitioning cost of $a \cdot n + O(n^{1-\varepsilon})$, for constants a and $\varepsilon > 0$, this recurrence has the following solution.

THEOREM 2.1. *Let A be a k -pivot quicksort algorithm that for each subarray of length n has partitioning cost $E(P_n) = a \cdot n + O(n^{1-\varepsilon})$ for a constant $\varepsilon > 0$. Then*

$$E(C_n) = \frac{1}{H_{k+1} - 1} \cdot an \ln n + O(n), \quad (3)$$

where $H_{k+1} = \sum_{i=1}^{k+1} (1/i)$ is the $(k+1)$ st harmonic number.

PROOF. By linearity of expectation we may solve the recurrence for partitioning cost $E(P_{1,n}) = a \cdot n$ and $E(P_{2,n}) = O(n^{1-\varepsilon})$ separately. To solve for cost $E(P_{1,n})$, we may apply Equation (1). Solving the recurrence for partitioning cost $E(P_{2,n})$ is a standard application of the continuous Master theorem from Roura [2001]; see Appendix A for details. $\square$

When focusing only on the average comparison count, it suffices to study the *classification problem*: Given a random permutation $(e_1, \dots, e_n)$ of $\{1, \dots, n\}$, choose the pivots $p_1, \dots, p_k$ and classify each of the remaining $n - k$ elements as belonging to one of the groups $A_0, \dots, A_k$.

Algorithmically, the classification of a single element x with respect to the pivots $p_1, \dots, p_k$ is done by using a *comparison tree* λ . A comparison tree is a binary search tree, where the leaf nodes are labeled $A_0, \dots, A_k$ from left to right and the inner nodes are labeled $p_1, \dots, p_k$ in inorder. Figure 2 depicts a comparison tree for five pivots. We denote the depth of the A_h leaf in comparison tree λ by $\text{depth}_\lambda(A_h)$. Classifying an element then means searching for this element in the search tree. The classification of the element is the label of the leaf reached in that way; the number of comparisons required is $\text{depth}_\lambda(A_h)$ if x belongs to group A_h .

A *classification strategy* is formally described as a *classification tree* as follows. A classification tree is a $(k+1)$ -way tree with a root and $n - k$ levels of inner nodes as well as one leaf level. Each inner node v has two labels: an index $i(v) \in \{k+1, \dots, n\}$ and a comparison tree $\lambda(v)$. The element $e_{i(v)}$ is classified using the comparison tree $\lambda(v)$. The $k+1$ edges out of a node are labeled $0, \dots, k$, respectively, representing the outcome of the classification as belonging to group $A_0, \dots, A_k$, respectively. On each of the $(k+1)^{n-k}$ paths, each index from $\{k+1, \dots, n\}$ occurs exactly once. An input $(e_1, \dots, e_n)$ determines a path in the classification tree in the obvious way: Sort the pivots and then use the classification tree to classify $e_{k+1}, \dots, e_n$. The classification of the input can then be read off from the nodes and edges along the path from the root to a leaf as determined by the input in the classification tree.

To fix more notation, for each node v , and for $h \in \{0, \dots, k\}$, we let a_h^v be the number of edges labeled “ h ” on the path from the root to v . Furthermore, let $C_{h,i}$ denote the random variable that counts the number of elements classified as belonging to group A_h , for $h \in \{0, \dots, k\}$, in the first i levels, for $i \in \{0, \dots, n-k\}$, that is, $C_{h,i} = a_h^v$ when v is the node on level i of the classification tree reached for an input. In many proofs, we will need that $C_{h,i}$ is not far from its expectation $a_h/(n-k-i)$. This would be a trivial consequence of the Chernoff bound if the classification of elements were independent. However, the probabilities of classifying elements to a specific group change according to classifications made before. We will use the *method of averaged bounded differences* to show concentration despite dependencies between tests.

LEMMA 2.2. *Let the pivots $p_1, \dots, p_k$ be fixed. Let $C_{h,i}$ be defined as above. Then for each h with $h \in \{0, \dots, k\}$ and for each i with $1 \leq i \leq n-k$, we have that*

$$\Pr(|C_{h,i} - E(C_{h,i})| > n^{2/3}) \leq 2\exp(-n^{1/3}/2).$$

PROOF. Fix an arbitrary $h \in \{0, \dots, k\}$. Define the indicator random variable

$$X_j = [\text{the element classified in level } j \text{ belongs to group } A_h].$$

Of course, $C_{h,i} = \sum_{1 \leq j \leq i} X_j$. We let $c_j := |\mathbb{E}(C_{h,i} \mid X_1, \dots, X_j) - \mathbb{E}(C_{h,i} \mid X_1, \dots, X_{j-1})|$. Using linearity of expectation we may calculate

$$\begin{aligned}
 c_j &= \left| \sum_{k=1}^j X_k + \sum_{k=j+1}^i \frac{a_h - C_{h,j}}{n-j-2} - \sum_{k=1}^{j-1} X_k - \sum_{k=j}^i \frac{a_h - C_{h,j-1}}{n-j-1} \right| \\
 &= \left| X_j + \sum_{k=j+1}^i \frac{a_h - C_{h,j-1} - X_j}{n-j-2} - \sum_{k=j}^i \frac{a_h - C_{h,j-1}}{n-j-1} \right| \\
 &= \left| X_j - X_j \cdot \frac{i-j}{n-j-2} + (a_h - C_{h,j-1}) \left(\frac{i-j}{n-j-2} - \frac{i-j+1}{n-j-1} \right) \right| \\
 &= \left| X_j \left(1 - \frac{i-j}{n-j-2} \right) - (a_h - C_{h,j-1}) \left(\frac{n-i-2}{(n-j-1)(n-j-2)} \right) \right| \\
 &\leq \max \left\{ \left| X_j \left(1 - \frac{i-j}{n-j-2} \right) \right|, \left| \frac{a_h - C_{h,j-1}}{n-j-1} \right| \right\} \leq 1.
 \end{aligned}$$

Now we may apply the following bound known as the method of averaged bounded differences (see Dubhashi and Panconesi [2009, Theorem 5.3]):

$$\Pr(|C_{h,i} - \mathbb{E}(C_{h,i})| > t) \leq 2 \exp \left(-\frac{t^2}{2 \sum_{j \leq i} c_j^2} \right).$$

This yields

$$\Pr(|C_{h,i} - \mathbb{E}(C_{h,i})| > n^{2/3}) \leq 2 \exp \left(\frac{-n^{4/3}}{2i} \right),$$

which is not larger than $2 \exp(-n^{1/3}/2)$. $\square$

3. THE AVERAGE COMPARISON COUNT FOR PARTITIONING

In this section, we will obtain a formula for the average comparison count of an arbitrary classification strategy. We make the following observations for all classification strategies: We need $k \log k = O(1)$ comparisons to sort $e_1, \dots, e_k$, that is, to determine the k pivots $p_1, \dots, p_k$ in order. If an element x belongs to group A_i , then it must be compared to p_i and p_{i+1} . (Of course, no real comparison takes place against p_0 and p_{k+1} .) On average, this leads to $2(1 - 1/(k+1))(n-k) + O(1)$ comparisons—regardless of the actual classification strategy.

For the following paragraphs, we fix a classification strategy, that is, a classification tree T . Furthermore, we let v be an arbitrary inner node of T .

If $e_{i(v)}$ belongs to group A_h , then exactly $\text{depth}_{\lambda(v)}(A_h)$ comparisons are made to classify this element. We let X_v^T denote the number of comparisons that take place in node v during classification. Let P_n^T be the random variable that counts the number of comparisons being made when classifying an input sequence $(e_1, \dots, e_n)$ using T , that is, $P_n^T = \sum_{v \in T} X_v^T$. For the average comparison count $\mathbb{E}(P_n^T)$ for partitioning we get:

$$\mathbb{E}(P_n^T) = \frac{1}{\binom{n}{k}} \sum_{1 \leq p_1 < p_2 < \dots < p_k \leq n} \mathbb{E}(P_n^T \mid p_1, \dots, p_k).$$

We define $p_{p_1, \dots, p_k}^v$ as the probability that node v is reached if the pivots are $p_1, \dots, p_k$. We may write:

$$\begin{aligned} \mathbb{E}(P_n^T \mid p_1, \dots, p_k) &= \sum_{v \in T} \mathbb{E}(X_v^T \mid p_1, \dots, p_k) \\ &= \sum_{v \in T} p_{p_1, \dots, p_k}^v \cdot \mathbb{E}(X_v^T \mid p_1, \dots, p_k, v \text{ reached}). \end{aligned} \quad (4)$$

For a comparison tree λ and group sizes $a'_0, \dots, a'_k$, we define the *cost* of λ on these group sizes as the number of comparisons it makes for classifying an input with these group sizes, that is,

$$\text{cost}^\lambda(a'_0, \dots, a'_k) = \sum_{0 \leq i \leq k} \text{depth}_\lambda(A_i) \cdot a'_i.$$

Furthermore, we define its average cost $c_{\text{avg}}^\lambda(a'_0, \dots, a'_k)$ as follows:

$$c_{\text{avg}}^\lambda(a'_0, \dots, a'_k) := \frac{\text{cost}^\lambda(a'_0, \dots, a'_k)}{\sum_{0 \leq i \leq k} a'_i}. \quad (5)$$

Under the assumption that node v is reached and that the pivots are $p_1, \dots, p_k$, the probability that the element $e_{i(v)}$ belongs to group A_h is exactly $(a_h - a_h^v)/(n - k - \text{level}(v))$ for each $h \in \{0, \dots, k\}$. (Note that this means that the order in which elements are classified is arbitrary, so we could actually use some fixed ordering.) Summing over all groups, we get

$$\mathbb{E}(X_v^T \mid p_1, \dots, p_k, v \text{ reached}) = c_{\text{avg}}^{\lambda(v)}(a_0 - a_0^v, \dots, a_k - a_k^v).$$

Plugging this into Equation (4) gives

$$\mathbb{E}(P_n^T \mid p_1, \dots, p_k) = \sum_{v \in T} p_{p_1, \dots, p_k}^v \cdot c_{\text{avg}}^{\lambda(v)}(a_0 - a_0^v, \dots, a_k - a_k^v). \quad (6)$$

Let Λ_k be the set of all possible comparison trees. For each $\lambda \in \Lambda_k$, we define the random variable F^λ that counts the number of times λ is used during classification. For given $p_1, \dots, p_k$, and for each $\lambda \in \Lambda_k$, we let

$$f_{p_1, \dots, p_k}^\lambda := \mathbb{E}(F^\lambda \mid p_1, \dots, p_k) = \sum_{\substack{v \in T \\ \lambda(v) = \lambda}} p_{p_1, \dots, p_k}^v$$

denote the average number of times comparison tree λ is used in T under the condition that the pivots are $p_1, \dots, p_k$.

Now, if it was decided in each step by independent random experiments with the correct expectation $a_h/(n - k)$, for $0 \leq h \leq k$, whether an element belongs to group A_h , then it would be clear that for each $\lambda \in \Lambda_k$ the contribution of λ to the average classification cost is $f_{p_1, \dots, p_k}^\lambda \cdot c_{\text{avg}}^\lambda(a_0, \dots, a_k)$. This intuition can be proven to hold for all classification trees, except that one gets an additional $O(n^{1-\varepsilon})$ term due to dependencies between classifications.

LEMMA 3.1. *Let the pivots $p_1, \dots, p_k$ be fixed. Let T be a classification tree. Then there exists a constant $\varepsilon > 0$ such that*

$$\mathbb{E}(P_n^T) = \frac{1}{\binom{n}{k}} \sum_{1 \leq p_1 < p_2 < \dots < p_k \leq n} \sum_{\lambda \in \Lambda_k} f_{p_1, \dots, p_k}^\lambda \cdot c_{\text{avg}}^\lambda(a_0, \dots, a_k) + O(n^{1-\varepsilon}).$$

PROOF. Fix the set of pivots $p_1, \dots, p_k$. The calculations start from re-writing Equation (6) in the following form:

$$\begin{aligned}
 \mathbb{E}(P_n^T \mid p_1, \dots, p_k) &= \sum_{v \in T} p_{p_1, \dots, p_k}^v \cdot c_{\text{avg}}^{\lambda(v)}(a_0 - a_0^v, \dots, a_k - a_k^v) \\
 &= \sum_{v \in T} p_{p_1, \dots, p_k}^v \cdot c_{\text{avg}}^{\lambda(v)}(a_0, \dots, a_k) \\
 &\quad - \sum_{v \in T} p_{p_1, \dots, p_k}^v \left(c_{\text{avg}}^{\lambda(v)}(a_0, \dots, a_k) - c_{\text{avg}}^{\lambda(v)}(a_0 - a_0^v, \dots, a_k - a_k^v) \right) \\
 &= \sum_{\lambda \in \Lambda_k} f_{p_1, \dots, p_k}^\lambda \cdot c_{\text{avg}}^\lambda(a_0, \dots, a_k) \\
 &\quad - \sum_{v \in T} p_{p_1, \dots, p_k}^v \left(c_{\text{avg}}^{\lambda(v)}(a_0, \dots, a_k) - c_{\text{avg}}^{\lambda(v)}(a_0 - a_0^v, \dots, a_k - a_k^v) \right). \quad (7)
 \end{aligned}$$

For each node v in the classification tree, we say that v is *on track* (to the expected values) if

$$|c_{\text{avg}}^{\lambda(v)}(a_0, \dots, a_k) - c_{\text{avg}}^{\lambda(v)}(a_0 - a_0^v, \dots, a_k - a_k^v)| \leq \frac{k^2}{n^{1/12}}.$$

Otherwise, v is called *off track*.

By considering on-track and off-track nodes in Equation (7) separately, we may calculate

$$\begin{aligned}
 \mathbb{E}(P_n^T \mid p_1, \dots, p_k) &\leq \sum_{\lambda \in \Lambda_k} f_{p_1, \dots, p_k}^\lambda \cdot c_{\text{avg}}^\lambda(a_0, \dots, a_k) + \sum_{\substack{v \in T \\ v \text{ is on track}}} p_{p_1, \dots, p_k}^v \frac{k^2}{n^{1/12}} \\
 &\quad + \sum_{\substack{v \in T \\ v \text{ is off track}}} p_{p_1, \dots, p_k}^v \left(c_{\text{avg}}^{\lambda(v)}(a_0, \dots, a_k) - c_{\text{avg}}^{\lambda(v)}(a_0 - a_0^v, \dots, a_k - a_k^v) \right) \\
 &\leq \sum_{\lambda \in \Lambda_k} f_{p_1, \dots, p_k}^\lambda \cdot c_{\text{avg}}^\lambda(a_0, \dots, a_k) + k \cdot \sum_{\substack{v \in T \\ v \text{ is off track}}} p_{p_1, \dots, p_k}^v + O(n^{11/12}) \\
 &= \sum_{\lambda \in \Lambda_k} f_{p_1, \dots, p_k}^\lambda \cdot c_{\text{avg}}^\lambda(a_0, \dots, a_k) \\
 &\quad + k \cdot \sum_{i=1}^{n-k} \Pr(\text{an off-track node is reached on level } i) + O(n^{11/12}). \quad (8)
 \end{aligned}$$

It remains to bound the second summand of Equation (8). First, we obtain the general bound:

$$\begin{aligned}
 &|c_{\text{avg}}^{\lambda(v)}(a_0, \dots, a_k) - c_{\text{avg}}^{\lambda(v)}(a_0 - a_0^v, \dots, a_k - a_k^v)| \\
 &\leq (k-1) \cdot \sum_{j=0}^k \left| \frac{a_j}{n-k} - \frac{a_j - a_j^v}{n-k - \text{level}(v)} \right| \\
 &\leq (k-1) \cdot (k+1) \cdot \max_{0 \leq j \leq k} \left\{ \left| \frac{a_j}{n-k} - \frac{a_j - a_j^v}{n-k - \text{level}(v)} \right| \right\}.
 \end{aligned}$$

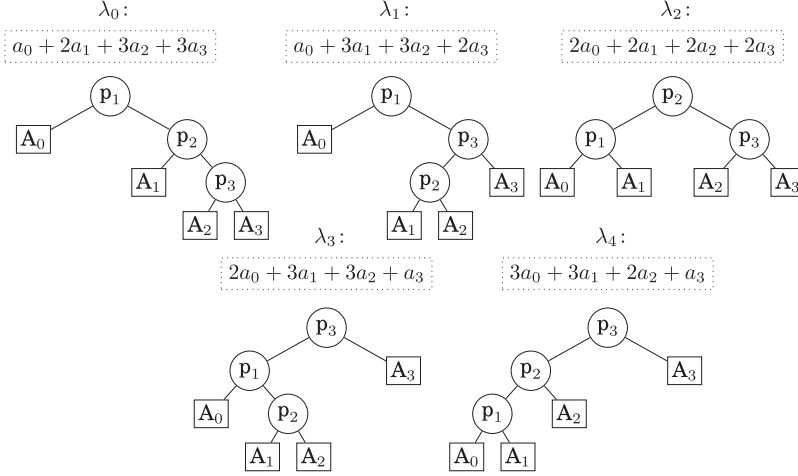


Fig. 3. The different comparison trees for three-pivot quicksort with their comparison cost (dotted boxes, only displaying the numerator).

Thus, by definition, whenever v is an off-track node, there exists $j \in \{0, \dots, k\}$ such that

$$\left| \frac{a_j}{n-k} - \frac{a_j - a_j^v}{n-k - \text{level}(v)} \right| > \frac{1}{n^{1/12}}.$$

Now consider the case that the random variables $C_{h,i}$ that count the number of A_h -elements in the first i classifications are concentrated around their expectation, as in the statement of Lemma 2.2. This happens with very high probability, so the contributions of the other case to the average comparison count can be neglected. For each $h \in \{0, \dots, k\}$, and each level $i \in \{1, \dots, n-k\}$, we calculate

$$\left| \frac{a_h}{n-k} - \frac{a_h - C_{h,i}}{n-k-i} \right| \leq \left| \frac{a_h}{n-k} - \frac{a_h(1-i/(n-k))}{n-k-i} \right| + \left| \frac{n^{2/3}}{n-k-i} \right| = \frac{n^{2/3}}{n-k-i}.$$

So, for the first $i \leq n - n^{3/4}$ levels, we are in an *on-track node* on level i with very high probability, because the deviation of the ideal probability $a_h/(n-k)$ of seeing an element that belongs to group A_h and the actual probability in the node reached on level i of seeing such an element is at most $1/n^{1/12}$. Thus, in total over the first $n - n^{3/4}$ levels, the contribution of the probabilities of off-track nodes to the second summand in Equation (8) is not more than $O(n^{11/12})$. For the last $n^{3/4}$ levels of the tree, we use that the contribution of the probabilities that we reach an off-track node on level i is at most 1 for a fixed level.

This shows that the second summand in Equation (8) is $O(n^{11/12})$. The lemma now follows from averaging over all possible pivot choices. $\square$

4. EXAMPLE: 3-PIVOT QUICKSORT

Here we study variants of three-pivot quicksort algorithms in the light of Lemma 3.1. This paradigm got recent attention by the work of Kushagra et al. [2014], who provided evidence that—in practice—a three-pivot quicksort algorithm might be faster than the YBB dual-pivot algorithm.

In three-pivot quicksort, we might choose from five different comparison trees. These trees, together with their comparison cost, are depicted in Figure 3. We will study the average comparison count of three different strategies in an artificial

setting: We assume, as in the analysis, that our input is a permutation of $\{1, \dots, n\}$. So, after choosing the pivots, the algorithm knows the exact group sizes in advance. Transforming this strategy into a realistic one is a topic of the next section.

All considered strategies will follow the same idea: After choosing the pivots, it is checked which comparison tree has the smallest average cost for the group sizes found in the input. Then this tree is used for all classifications. Our strategies differ with respect to the set of comparison trees they can use. In the next section, we will explain why deviating from such a strategy, that is, using different trees during the classification for fixed group sizes, does not help for minimizing the average comparison count.

The Symmetric Strategy. In the algorithm of Kushagra et al. [2014], the balanced comparison tree λ_2 is used for each classification. Using Lemma 3.1, we get¹

$$\begin{aligned} E(P_n) &= \frac{1}{\binom{n}{3}} \sum_{a_0+a_1+a_2+a_3=n-3} (2a_0 + 2a_1 + 2a_2 + 2a_3) + O(n^{1-\varepsilon}) \\ &= 2n + O(n^{1-\varepsilon}). \end{aligned}$$

Using Theorem 2.1, we conclude that

$$E(C_n) = (24/13)n \ln n + O(n) \approx 1.846n \ln n + O(n),$$

as known from Kushagra et al. [2014]. This improves on classical quicksort ($2n \ln n + O(n)$ comparisons on average) but is worse than optimal dual-pivot quicksort ($1.8n \ln n + O(n)$ comparisons on average [Aumüller and Dietzfelbinger 2015]) or median-of-three quicksort ($1.714n \ln n + O(n)$ comparisons on average [van Emden 1970]).

Using Three Trees. Here we restrict our algorithm to choose only among the comparison trees $\lambda_1, \lambda_2, \lambda_3$. The computation of a cost-minimal comparison tree is then simple: Suppose that the segment sizes are $a_0, \dots, a_3$. If $a_0 > a_3$ and $a_0 > a_1 + a_2$, then comparison tree λ_1 has minimum cost. If $a_3 \geq a_0$ and $a_3 > a_1 + a_2$, then comparison tree λ_3 has minimum cost. Otherwise, λ_2 has minimum cost.

Using Lemma 3.1, the average partition cost with respect to this set of comparison trees can be calculated (using Maple) as follows:

$$\begin{aligned} E(P_n) &= \frac{1}{\binom{n}{3}} \sum_{a_0+a_1+a_2+a_3=n-3} \min\left\{ \begin{matrix} a_0+3a_1+3a_2+2a_3, & 2a_0+2a_1+2a_2+2a_3, \\ & 2a_0+3a_1+3a_2+a_3 \end{matrix} \right\} + O(n^{1-\varepsilon}) \\ &= \frac{17}{9}n + O(n^{1-\varepsilon}). \end{aligned}$$

This yields the following average comparison cost:

$$E(C_n) = \frac{68}{39}n \ln n + O(n) \approx 1.744n \ln n + O(n).$$

Using All Trees. Now we let our strategies choose among all five trees. Using Lemma 3.1 and the average cost for all trees from Figure 3, we calculate (using Maple)

¹Of course, $E(P_n) = 2(n-3)$, since each classification makes exactly two comparisons.

$$\begin{aligned}
E(P_n) &= \frac{1}{\binom{n}{3}} \sum_{a_0+a_1+a_2+a_3=n-3} \min \left\{ \begin{matrix} a_0+2a_1+3a_2+3a_3, a_0+3a_1+3a_2+2a_3, \\ 2a_0+2a_1+2a_2+2a_3, 2a_0+3a_1+3a_2+a_3 \\ 3a_0+3a_1+2a_2+a_3 \end{matrix} \right\} + O(n^{1-\varepsilon}) \\
&= \frac{133}{72}n + O(n^{1-\varepsilon}).
\end{aligned} \tag{9}$$

This yields the following average comparison cost:

$$E(C_n) = \frac{133}{78}n \ln n + O(n) \approx 1.705n \ln n + O(n),$$

which is—as will be explained in the next section—the lowest possible average comparison count one can achieve by picking three pivots directly from the input. So, using three pivots gives a slightly lower average comparison count than quicksort using the median of three elements as the pivot.

5. (ASYMPTOTICALLY) OPTIMAL CLASSIFICATION STRATEGIES

In this section, we will discuss four different strategies, which will all achieve the minimal average comparison count (up to lower order terms). Two of these four strategies will be optimal but unrealistic, since they assume that after the pivots are fixed the algorithm knows the sizes of the $k+1$ different groups. The strategies work as follows: One strategy maintains the group sizes of the unclassified part of the input and chooses the comparison tree with minimum cost with respect to these group sizes. This will turn out to be the optimal classification strategy for k -pivot quicksort. To turn this strategy into an actual algorithm, we will use the group sizes of the already classified part of the input as a basis for choosing the comparison tree for the next classification. The second unrealistic strategy works like the algorithms for three-pivot quicksort. It will use the comparison tree with minimum cost with respect to the group sizes of the input in each classification. To get an actual algorithm, we estimate these group sizes in a small sampling step. Note that these strategies are the obvious generalization of the optimal strategies for dual-pivot quicksort from Aumüller and Dietzfelbinger [2015].

Since all these strategies need to compute cost-minimal comparison trees, this section starts with a short discussion of algorithms for this problem. Then we discuss the four different strategies.

5.1. Choosing an Optimal Comparison Tree

For optimal k -pivot quicksort algorithms, it is of course necessary to devise an algorithm that can compute an optimal comparison tree for partition sizes $a_0, \dots, a_k$, that is, a comparison tree that minimizes Equation (5). It is well known that the number of binary search trees with k inner nodes equals the k th Catalan number, which is approximately $4^k / ((k+1)\sqrt{\pi k})$. Choosing an optimal comparison tree is a standard application of dynamic programming and is known from textbooks as “choosing an optimum binary search tree,” see, for example, Knuth [1973]. The algorithm runs in time and space $O(k^2)$. In our case where the cost is only associated with the leaves, we wish to find an optimal *alphabetic* binary tree. This can be solved in time $O(k \log k)$ using either the algorithm of Hu and Tucker [1971] or that of Garsia and Wachs [1977].

5.2. The Optimal Classification Strategy and Its Algorithmic Variant

Here, we consider the following strategy² $\mathcal{O}_k$: *Given $a_0, \dots, a_k$, the comparison tree $\lambda(v)$ is one that minimizes $\text{cost}^\lambda(a_0 - a_0^v, \dots, a_k - a_k^v)$ over all comparison trees λ .*

²For all strategies, we say which comparison tree is used in a given node of the classification tree. Recall that the classification order is arbitrary.

While being unrealistic, since the exact partition sizes $a_0, \dots, a_k$ are in general unknown to the algorithm, strategy $\mathcal{O}_k$ is the optimal classification strategy, that is, it minimizes the average comparison count.

THEOREM 5.1. *Strategy $\mathcal{O}_k$ is optimal for each k .*

PROOF. Strategy $\mathcal{O}_k$ chooses for each node v in the classification tree the comparison tree that minimizes the average cost in Equation (6). So it minimizes each term of the sum and thus minimizes the whole sum in Equation (6). $\square$

There exist other strategies whose average comparison count differs by at most $O(n^{1-\varepsilon})$ from the average comparison count of $\mathcal{O}_k$. We call such strategies *asymptotically optimal*.

Strategy $\mathcal{C}_k$ is an algorithmic variant of $\mathcal{O}_k$. It works as follows: *The comparison tree $\lambda(v)$ is one that minimizes $\text{cost}^\lambda(a_0^v, \dots, a_k^v)$ over all comparison trees λ .*

THEOREM 5.2. *Strategy $\mathcal{C}_k$ is asymptotically optimal for each k .*

PROOF. Since the average comparison count is independent of the actual order in which elements are classified, assume that strategy $\mathcal{O}_k$ classifies elements in the order $e_{k+1}, \dots, e_n$, while strategy $\mathcal{C}_k$ classifies them in reversed order, that is, $e_n, \dots, e_{k+1}$. Then the comparison tree that is used by $\mathcal{C}_k$ for element e_i is the one that $\mathcal{O}_k$ is using for element e_{i+1} because both strategies use the group sizes in $(e_{i+1}, \dots, e_n)$. Let P_i and P'_i denote the number of comparisons for the classification of the element e_{k+i} using strategy $\mathcal{O}_k$ and $\mathcal{C}_k$, respectively.

Fix some integer $i \in \{1, \dots, n-k\}$. Suppose that the input has group sizes $a_0, \dots, a_k$. Assume that the sequence $(e_{k+1}, \dots, e_i)$ contains a'_h elements of group A_h for $h \in \{0, \dots, k\}$, where $|a'_h - i \cdot a_h / (n-k)| \leq n^{2/3}$ for each $h \in \{0, \dots, k\}$. Let λ be a comparison tree with minimal cost w.r.t. $(a_0 - a'_0, \dots, a_k - a'_k)$. For a random input having group sizes from above we calculate:

$$\begin{aligned} |\mathbb{E}(P_{i+1}) - \mathbb{E}(P'_i)| &= |c_{\text{avg}}^\lambda(a_0 - a'_0, \dots, a_k - a'_k) - c_{\text{avg}}^\lambda(a'_0, \dots, a'_k)| \\ &= \left| \frac{\sum_{h=0}^k \text{depth}_\lambda(A_h) \cdot (a_h - a'_h)}{n-k-i} - \frac{\sum_{h=0}^k \text{depth}_\lambda(A_h) \cdot a'_h}{i} \right| \\ &\leq \sum_{h=0}^k \text{depth}_\lambda(A_h) \left| \frac{a_h - a'_h}{n-k-i} - \frac{a'_h}{i} \right| \\ &\leq \sum_{h=0}^k \text{depth}_\lambda(A_h) \left(\left| \frac{a_h - \frac{i \cdot a_h}{n-k}}{n-k-i} - \frac{i \cdot a_h}{i} \right| + \frac{n^{2/3}}{n-k-i} + \frac{n^{2/3}}{i} \right) \\ &= \sum_{h=0}^k \text{depth}_\lambda(A_h) \left(\frac{n^{2/3}}{n-k-i} + \frac{n^{2/3}}{i} \right) \leq \frac{k^2 \cdot n^{2/3}}{n-k-i} + \frac{k^2 \cdot n^{2/3}}{i}. \end{aligned}$$

Assume that the concentration argument of Lemma 2.2 holds. Then the difference between the average comparison count for element e_{i+1} (for $\mathcal{O}_k$) and e_i (for $\mathcal{C}_k$) is at most

$$\frac{k^2 \cdot n^{2/3}}{n-k-i} + \frac{k^2 \cdot n^{2/3}}{i}.$$

The difference of the average comparison count over all elements $e_i, \dots, e_j, i \geq n^{3/4}, j \leq n - n^{3/4}$, is then at most $O(n^{11/12})$. For elements that reside outside of this range, the difference in the average comparison count is at most $2n^{3/4} \cdot k$. Furthermore, error terms

for cases where the concentration argument does not hold can be neglected because they occur with exponentially low probability. So, the total difference of the average comparison count between strategy $\mathcal{O}_k$ and strategy $\mathcal{C}_k$ is at most $O(n^{11/12})$. $\square$

This shows that the optimal strategy $\mathcal{O}_k$ can be approximated by an actual algorithm that makes an error of up to $O(n^{11/12})$, which sums up to an error term of $O(n)$ over the whole recursion by Theorem 2.1. In the case of dual-pivot quicksort, the difference between $\mathcal{O}_2$ and $\mathcal{C}_2$ is $O(\log n)$, which also sums up to a difference of $O(n)$ over the whole recursion [Aumüller and Dietzfelbinger 2015]. It remains an open question to prove tighter bounds than $O(n^{11/12})$ in the general case.

5.3. A Fixed Strategy and Its Algorithmic Variant

Now we turn to strategy $\mathcal{N}_k$: Given $a_0, \dots, a_k$, the comparison tree $\lambda(v)$ used at node v is one that minimizes $\text{cost}^\lambda(a_0, \dots, a_k)$ over all comparison trees λ .

Strategy $\mathcal{N}_k$ uses a fixed comparison tree for all classifications for given partition sizes, but it has to know these sizes in advance.

THEOREM 5.3. *Strategy $\mathcal{N}_k$ is asymptotically optimal for each k .*

PROOF. According to Lemma 3.1, the average comparison count is determined up to lower-order terms by the parameters $f_{p_1, \dots, p_k}^\lambda$, for each $\lambda \in \Lambda_k$. For each $p_1, \dots, p_k$, strategy $\mathcal{N}_k$ chooses the comparison tree that minimizes the average cost. By Lemma 3.1, this is optimal up to an $O(n^{1-\varepsilon})$ term. $\square$

We will now describe how to implement strategy $\mathcal{N}_k$ by using sampling. Strategy $\mathcal{SP}_k$ works as follows: Let $\lambda_0 \in \Lambda_k$ be an arbitrary comparison tree. After the pivots are chosen, inspect the first $n^{3/4}$ elements and classify them using λ_0 . Let $a'_0, \dots, a'_k$ denote the number of elements that belonged to $A_0, \dots, A_k$, respectively. Let λ be a comparison tree with minimal cost for $a'_0, \dots, a'_k$. Then classify each of the remaining elements by using λ .

THEOREM 5.4. *Strategy $\mathcal{SP}_k$ is asymptotically optimal for each k .*

PROOF. Fix the k pivots $p_1, \dots, p_k$ and thus $a_0, \dots, a_k$. According to Lemma 3.1, the average comparison count $E(P_n^{\mathcal{SP}_k} \mid p_1, \dots, p_k)$ can be calculated as follows:

$$E(P_n^{\mathcal{SP}_k} \mid p_1, \dots, p_k) = \sum_{\lambda \in \Lambda_k} f_{p_1, \dots, p_k}^\lambda \cdot c_{\text{avg}}^\lambda(a_0, \dots, a_k) + O(n^{1-\varepsilon}).$$

Let λ^* be a comparison tree with minimal cost w.r.t. $a_0, \dots, a_k$. Let $a'_0, \dots, a'_k$ be the partition sizes after inspecting $n^{3/4}$ elements. Let λ be a comparison tree with minimal cost w.r.t. $a'_0, \dots, a'_k$. We call λ *good* if

$$\begin{aligned} c_{\text{avg}}^\lambda(a_0, \dots, a_k) - c_{\text{avg}}^{\lambda^*}(a_0, \dots, a_k) &\leq \frac{2k}{n^{1/12}}, \text{ or equivalently} \\ \text{cost}^\lambda(a_0, \dots, a_k) - \text{cost}^{\lambda^*}(a_0, \dots, a_k) &\leq 2kn^{11/12}, \end{aligned} \quad (10)$$

otherwise we call λ *bad*. We define good_λ and bad_λ as the events that the sample yields a good and bad comparison tree, respectively.

We calculate:

$$\begin{aligned}
\mathbb{E}(P_n^{\mathcal{SP}_k} \mid p_1, \dots, p_k) &= \sum_{\substack{\lambda \in \Lambda_k \\ \lambda \text{ good}}} f_{p_1, \dots, p_k}^\lambda \cdot c_{\text{avg}}^\lambda(a_0, \dots, a_k) \\
&\quad + \sum_{\substack{\lambda \in \Lambda_k \\ \lambda \text{ bad}}} f_{p_1, \dots, p_k}^\lambda \cdot c_{\text{avg}}^\lambda(a_0, \dots, a_k) + O(n^{1-\varepsilon}) \\
&\leq n \cdot c_{\text{avg}}^{\lambda^*}(a_0, \dots, a_k) + \sum_{\substack{\lambda \in \Lambda_k \\ \lambda \text{ bad}}} f_{p_1, \dots, p_k}^\lambda \cdot c_{\text{avg}}^\lambda(a_0, \dots, a_k) + O(n^{1-\varepsilon}) \\
&\leq n \cdot c_{\text{avg}}^{\lambda^*}(a_0, \dots, a_k) + k \cdot \sum_{\substack{\lambda \in \Lambda_k \\ \lambda \text{ bad}}} f_{p_1, \dots, p_k}^\lambda + O(n^{1-\varepsilon}). \tag{11}
\end{aligned}$$

Now we derive an upper bound for the second summand of Equation (11). After the first $n^{3/4}$ classifications, the algorithm will either use a good comparison tree or a bad comparison tree for the remaining classifications. The probability $\Pr(\text{bad}_\lambda \mid p_1, \dots, p_k)$ is the ratio of nodes on each level from $n^{3/4}$ to $n - k$ of the classification tree of nodes labeled with bad trees (in the sense of Equation (10)). Summing over all levels, the second summand of Equation (11) is thus at most $k \cdot n \cdot \Pr(\text{bad}_\lambda \mid p_1, \dots, p_k) + O(n^{3/4})$, where the latter summand collects error terms for the first $n^{3/4}$ steps.

LEMMA 5.5. *Conditioned on $p_1, \dots, p_k$, good_λ occurs with very high probability.*

PROOF. For each $i \in \{0, \dots, k\}$, let a'_i be the random variable that counts the number of elements from the sample that belong to group A_i . According to Lemma 2.2, with very high probability we have that $|a'_i - \mathbb{E}(a'_i)| \leq n^{2/3}$ for each i with $0 \leq i \leq k$. By the union bound, with very high probability there is no a'_i that deviates by more than $n^{2/3}$ from its expectation $n^{-1/4} \cdot a_i$. We will now show that if this happens then the event good_λ occurs. We obtain the following upper bound for an arbitrary comparison tree $\lambda' \in \Lambda_k$:

$$\begin{aligned}
\text{cost}^{\lambda'}(a'_0, \dots, a'_k) &= \sum_{0 \leq i \leq k} \text{depth}_{\lambda'}(A_i) \cdot a'_i \\
&\leq \sum_{0 \leq i \leq k} \text{depth}_{\lambda'}(A_i) \cdot n^{2/3} + n^{-1/4} \cdot \text{cost}^{\lambda'}(a_0, \dots, a_k) \\
&\leq k^2 n^{2/3} + n^{-1/4} \cdot \text{cost}^{\lambda'}(a_0, \dots, a_k).
\end{aligned}$$

Similarly, we get a corresponding lower bound. Thus, for each comparison tree $\lambda' \in \Lambda_k$, it holds that

$$\frac{\text{cost}^{\lambda'}(a_0, \dots, a_k)}{n^{1/4}} - k^2 n^{2/3} \leq \text{cost}^{\lambda'}(a'_0, \dots, a'_k) \leq \frac{\text{cost}^{\lambda'}(a_0, \dots, a_k)}{n^{1/4}} + k^2 n^{2/3},$$

and we get the following bound:

$$\begin{aligned}
\text{cost}^\lambda(a_0, \dots, a_k) - \text{cost}^{\lambda^*}(a_0, \dots, a_k) &\leq n^{1/4} (\text{cost}^\lambda(a'_0, \dots, a'_k) - \text{cost}^{\lambda^*}(a'_0, \dots, a'_k)) + 2n^{1/4} \cdot k^2 \cdot n^{2/3} \\
&\leq 2k^2 \cdot n^{11/12}.
\end{aligned}$$

(The last inequality follows because λ has minimal cost w.r.t. $a'_0, \dots, a'_k$.) Hence, λ is good. $\square$

Table I. Optimal Average Comparison Count for k -Pivot Quicksort for $k \in \{2, \dots, 9\}$ Note That the Values for $k \geq 4$ Are Based on Experiments. For Odd k , we Also Include the Average Comparison Count of Quicksort with the Median-of- k Strategy. (The Numbers for the Median-of- k Variant Can Be Found in van Emden [1970] or Hennequin [1991])

Pivot Number k	opt. k -pivot	median-of- k
2	$1.800n \ln n$	—
3	$1.705n \ln n$	$1.714n \ln n$
4	$1.650n \ln n$	—
5	$1.610n \ln n$	$1.622n \ln n$
6	$1.590n \ln n$	—
7	$1.577n \ln n$	$1.576n \ln n$
8	$1.564n \ln n$	—
9	$1.555n \ln n$	$1.549n \ln n$

Thus, the average comparison count of $\mathcal{SP}_k$ is at most a summand of $O(n^{1-\epsilon})$ larger than the average comparison count of $\mathcal{N}_k$. This implies that $\mathcal{SP}_k$ is asymptotically optimal as well. $\square$

Since the number of comparison trees in Λ_k is exponentially large in k , one might want to restrict the set of used comparison trees to some subset $\Lambda'_k \subseteq \Lambda_k$. We remark here that our strategies are optimal w.r.t. any chosen subset of comparison trees as well.

6. THE OPTIMAL AVERAGE COMPARISON COUNT OF K -PIVOT QUICKSORT

In this section, we use the theory developed so far to discuss the optimal average comparison count of k -pivot quicksort. We compare the result to the well-known median-of- k strategy of classical quicksort [van Emden 1970].

By Lemma 3.1 and Theorem 5.3, the minimal partitioning cost for k -pivot quicksort (up to lower order terms) is

$$\frac{1}{\binom{n}{k}} \sum_{a_0 + \dots + a_k = n-k} \min\{\text{cost}^\lambda(a_0, \dots, a_k) \mid \lambda \in \Lambda_k\} + O(n^{1-\epsilon}). \quad (12)$$

Applying Theorem 2.1 then gives the minimal average comparison count for k -pivot quicksort.

Unfortunately, we were not able to solve Equation (12) for $k \geq 4$. (The solution for $k = 3$ as stated in Section 4 already required a lot of manual tweaking before using Maple.) This remains an open question. We resorted to experiments. As noticed in Aumüller and Dietzfelbinger [2015], estimating the total average comparison count by sorting inputs does not allow us to estimate the leading term of the average comparison count correctly, because the $O(n)$ term in Equation (1) has a big influence on the average comparison count for real-world input lengths. We used the following approach instead: For $n = 50 \cdot 10^6$, we generated 10 000 random permutations of $\{1, \dots, n\}$ and ran strategy $\mathcal{O}_k$ for each input, that is, only classified the input.³ For the average partitioning cost measured in these experiments, we then applied Equation (1) to derive the leading factor of the total average comparison count. Table I shows the results from these experiments for $k \in \{2, \dots, 9\}$. Note that the results for $k \in \{2, 3\}$ are almost identical to the exact theoretical results. Additionally, the table shows the theoretical results known for classical quicksort using the median-of- k strategy [van Emden 1970; Hennequin 1991]. Interestingly, from Table I we see that—based on our

³Experiments with other input sizes gave exactly the same results.

experimental data for k -pivot quicksort—starting from $k = 7$ the median-of- k strategy has a slightly lower average comparison count than the (rather complicated) optimal partitioning methods for k -pivot quicksort.

7. REARRANGING ELEMENTS

With this section, we change our viewpoint on multi-pivot quicksort in two respects: we consider cost measures that differ from comparisons and focus on one particularly interesting algorithm for the “rearrangement problem.” The goal now is to find other cost measures that show differences in multi-pivot quicksort algorithms with respect to running time in practice.

7.1. Which Factors Are Relevant for Running Time?

Let us first reflect on the influence of key comparisons to running time. From a running-time perspective it seems unintuitive that comparisons are the crucial factor with regard to running time, especially when key comparisons are cheap, for example, when comparing 32-bit integers. However, while a comparison is often cheap, mispredicting the destination that a branch takes, that is, the outcome of the comparison, may incur a significant penalty in running time, because the CPU wasted work on executing instructions on the wrongly predicted branch. One famous example for the effect of branch prediction is Kaligosi and Sanders [2006] in which quicksort is made faster by choosing a skewed pivot due to pipelining effects on a certain CPU. In very recent work, Martínez et al. [2015] considered differences in branch misses between classical quicksort and the YBB algorithm but found no crucial differences. They concluded that the advantages in running time of the dual-pivot approach are not due to differences in branch prediction.

Traditionally, the *cost of moving elements around* is also considered as a cost measure of sorting algorithms. This cost is usually expressed as the number of *swap* operations or the number of *assignments* needed to sort the input. Kushagra et al. [2014] take a different approach and investigate quicksort variants with respect to their *cache behavior*. How well an algorithm makes use of the CPU cache(s) is important for its practical performance because an access to main memory in modern computers can be slower than executing a few hundred simple CPU instructions. Caches speed these accesses up, but their influence seems difficult to analyze. Let us exemplify the influence of caches on running time. First, the cache structure of modern CPUs is usually hierarchical. For example, the Intel i7 that we used in our experiments has three data caches: There is a very small L1 cache (32KB of data) and a slightly larger L2 cache (256KB of data) very close to the processor. Each CPU core has its own L1 and L2 cache. They are both eight-way associative, that is, a memory segment can be stored at eight different cache lines. Shared among cores is a rather big L3 cache that can hold 8MB of data and is 16-way associative. Caches greatly influence running time. While a lookup in main memory costs many CPU cycles (≈ 140 cycles on the Intel i7 used in our experiments), a cache access is very cheap and costs about 4, 11, and 25 cycles for a hit in L1, L2, and L3 cache, respectively [Levinthal 2009]. Also, modern CPUs use *prefetching* to load memory segments into cache before they are accessed. Usually, there exist different prefetchers for different caches, and there exist different strategies to prefetch data, for example, “load two adjacent cache lines” or “load memory segments based on predictions by monitoring data flow.”

From a theoretical point of view, much research has been conducted to study algorithms with respect to their cache behavior, see, for example, the survey article of Rahman [2002]. (We recommend this article as an excellent introduction to the topic of caches.)

In Kushagra et al. [2014], a fast three-pivot algorithm was described. They analyzed its cache behavior and compared it to classical quicksort and Yaroslavsky's dual-pivot quicksort algorithm using the approach of LaMarca and Ladner [1999]. Their results gave reason to believe that the improvements of multi-pivot quicksort algorithms with respect to running times result from their better cache behavior. They also reported from experiments with a seven-pivot algorithm, which ran more slowly than their three-pivot algorithm. Very recently, Nebel et al. [2016] gave a more detailed analysis of the cache behavior of the YBB algorithm also with respect to different sampling strategies. An important contribution of Nebel et al. [2016] is the distinction of a theoretical measure *scanned elements* (basically the number of times a memory cell is inspected during sorting) and the usage of this cost measures to predict cache behavior.

In this section, we discuss how the considerations of Kushagra et al. [2014] and Nebel et al. [2016] generalize to the case of using more than three pivots. In connection with the running-time experiments from Section 9, this allows us to make more accurate predictions than in Kushagra et al. [2014] about the influence of cache behavior on running time. One result of this study will be that it is not surprising that their seven-pivot approach is slower, because it has worse cache behavior than three- or five-pivot quicksort algorithms using a specific partitioning strategy.

We will start by specifying the problem setting and subsequently introduce a generalized partitioning algorithm for k pivots. This algorithm is the generalization of the partitioning methods used in classical quicksort, the YBB algorithm, and the three-pivot quicksort algorithm of Kushagra et al. [2014]. This strategy will be evaluated for different values of k with respect to different memory-related cost measures, which will be introduced later. It will turn out that these theoretical cost measures allow us to give detailed recommendations under which circumstances a multi-pivot quicksort approach has advantages over classical quicksort.

We remark that the same analysis can be done for other partitioning algorithms. In his Ph.D. thesis, Aumüller [2015] considers two variants of the super scalar sample sort algorithm of Sanders and Winkel [2004] according to the same cost measures that are studied here. We give a short overview of the results at the end of this section.

7.2. The Rearrangement Problem

With regard to counting key comparisons, we defined the classification problem to abstract from the situation that a multi-pivot quicksort algorithm has to move elements around to produce the partition. Here, we assume that for each element its groups is known, and we are only interested in moving elements around to produce the partition. This motivates us to consider the *rearrangement problem for k pivots*: Given a sequence of length $n - k$ with entries having labels from the set $\{A_0, \dots, A_k\}$ of group names, the task is to rearrange the entries with respect to their labels into ascending order, where $A_i < A_{i+1}$ for $i \in \{0, \dots, k - 1\}$. Note that any classification strategy can be used to find out element groups. We assume that the input resides in an array $A[1..n]$ where the k first cells hold the pivots.⁴ For $k = 2$, this problem is known under the name *Dutch national flag problem*, proposed by Dijkstra [1976]. For $k > 2$, the problem was considered in the article of McIlroy et al. [1993], who devised an algorithm called “American flag sort” to solve the rearrangement problem for $k > 2$. We will discuss the applicability of these algorithms at the end of this section. Our goal is to analyze algorithms for this problem with respect to different cost measures, for example, the number of array cells that are inspected during rearranging the input or the number

⁴We shall disregard the pivots in the description of the problem. In a final step, the k pivots have to be moved into the correct positions between group segments. This is possible by moving not more than k^2 elements around using k rotate operations, as introduced below.

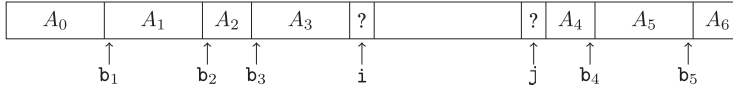


Fig. 4. General memory layout of Algorithm 1 for $k = 6$. Two pointers i and j are used to scan the array from left to right and right to left, respectively. Pointers $b_1, \dots, b_{k-1}$ are used to point to the start (respectively, end) of segments.

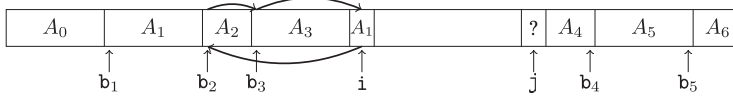


Fig. 5. The rotate operation in Line 9 of Algorithm 1. An element that belongs to group A_1 is moved into its respective segment. Pointers i, b_2, b_3 are increased by 1 afterwards.

of times the algorithm writes to array cells in the process. We start by introducing an algorithm for the rearrangement problem that generalizes the algorithmic ideas behind rearranging elements in classical quicksort, the YBB algorithm [Nebel et al. 2016], and the three-pivot algorithm of Kushagra et al. [2014].

7.3. The Algorithm

To capture the cost of rearranging the elements, in the analysis of sorting algorithms one traditionally uses the “swap”-operation, which exchanges two elements. The cost of rearranging is then the number of swap operations performed during the sorting process. In the case where one uses two or more pivots, we will see that it is beneficial to generalize this operation. We define the operation $\text{rotate}(i_1, \dots, i_\ell)$ as follows:

$$\text{tmp} \leftarrow A[i_1]; A[i_1] \leftarrow A[i_2]; A[i_2] \leftarrow A[i_3]; \dots; A[i_{\ell-1}] \leftarrow A[i_\ell]; A[i_\ell] \leftarrow \text{tmp}.$$

The operation rotate performs a cyclic shift of the elements by one position. A $\text{swap}(A[i_1], A[i_2])$ is a $\text{rotate}(i_1, i_2)$. A $\text{rotate}(i_1, \dots, i_\ell)$ operation makes exactly $\ell + 1$ assignments and inspects and writes into ℓ array cells.

For each $k \geq 1$, we consider an algorithm Exchange_k . Pseudocode of this algorithm is given in Algorithm 1. The basic idea is similar to classical quicksort: Two pointers⁵ scan the array. One pointer scans the array from left to right; another pointer scans the array from right to left, exchanging misplaced elements on the way. Formally, the algorithm uses two pointers i and j . At the beginning, i points to the element in $A[k+1]$ and j points to the element in $A[n]$. We set $m = \lceil \frac{k+1}{2} \rceil$. The algorithm makes sure that all elements to the left of pointer i belong to groups $A_0, \dots, A_{m-1}$ (and are arranged in this order), that is, m is the number of groups left of pointer i . Also, all elements to the right of pointer j belong to groups $A_m, \dots, A_k$, arranged in this order. To do so, Algorithm 1 uses $k - 1$ additional “border pointers” $b_1, \dots, b_{k-1}$. For $i < m$, the algorithm makes sure that at each point in time, pointer b_i points to the leftmost element to the left of pointer i that belongs to group $A_{i'}$ with $i' \geq i$. Analogously, for $j \geq m$, the algorithm makes sure that pointer b_j points to the rightmost element to the right of pointer j that belongs to group $A_{j'}$ with $j' \leq j$, see Figure 4. As long as pointers i and j have not crossed yet, the algorithm increments pointer i until i points to an element that belongs to a group A_p with $p \geq m$. For each element x along the way that belongs to a group $A_{p'}$ with $p' < m - 1$, it moves x to the place to which $b_{p'+1}$ points, using a rotate operation to make space to accommodate the element, see Figure 5 and Lines 7–11 in Algorithm 1. Pointers $b_{p'+1}, \dots, b_{m-1}$ are incremented afterwards. Similarly, the

⁵Note that our pointers are actually variables that hold an array index.

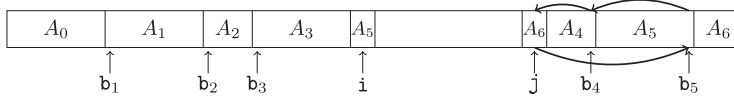


Fig. 6. The rotate operation in Line 14 of Algorithm 1. An element that belongs to group A_6 is moved into its respective segment. Pointers j , b_4 , b_5 are decreased by 1 afterwards.

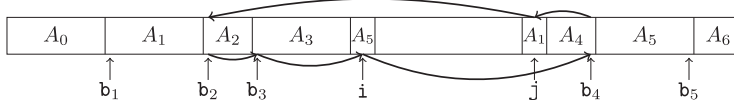


Fig. 7. Example for the rotate operation in Line 18 of Algorithm 1. The element found at i is moved into its specific segment. Subsequently, the element found at j is moved into its specific segment.

ALGORITHM 1: Move Elements by Rotations to Produce a Partition

procedure $Exchange_k(A[1..n])$

```

1:  $i \leftarrow k + 1; j \leftarrow n;$ 
2:  $m \leftarrow \lceil \frac{k+1}{2} \rceil;$ 
3:  $b_1, \dots, b_{m-1} \leftarrow i;$ 
4:  $b_m, \dots, b_{k-1} \leftarrow j;$ 
5:  $p, q \leftarrow -1;$   $\triangleright p$  and  $q$  hold the group indices of the elements indexed by  $i$  and  $j$ .
6: while  $i < j$  do
7:   while  $A[i]$  belongs to group  $A_p$  with  $p < m$  do
8:     if  $p < m - 1$  then
9:        $rotate(i, b_{m-1}, \dots, b_{p+1});$ 
10:       $b_{p+1}++; \dots; b_{m-1}++;$ 
11:       $i++;$ 
12:   while  $A[j]$  belongs to group  $A_q$  with  $q \geq m$  do
13:     if  $q \geq m + 1$  then
14:        $rotate(j, b_m, \dots, b_{q-1});$ 
15:       $b_{q-1}--; \dots; b_m--;$ 
16:       $j--;$ 
17:   if  $i < j$  then
18:      $rotate(i, b_{m-1}, \dots, b_{q+1}, j, b_m, \dots, b_{p-1});$ 
19:      $i++; b_{q+1}++; \dots; b_{m-1}++;$ 
20:      $j--; b_m--; \dots; b_{p-1}--;$ 

```

algorithm decrements pointer j until it points to an element that belongs to a group A_q with $q < m$, moving elements from $A_{m+1}, \dots, A_k$ along the way in a similar fashion, see Figure 6 and Line 12–16 in Algorithm 1. If now $i < j$, then a single rotate operation suffices to move the elements referenced by i and j to a (temporarily) correct position, see Figure 7 and Line 17–20 in Algorithm 1. Note that any classification strategy can be used in an “online fashion” to find out element groups in Algorithm 1.

Figure 4 shows the idea of the algorithm for $k = 6$; Figures 5–7 show the different rotations being made by Algorithm 1 in lines 9, 14, and 18.

7.4. Cost Measures and Assumptions of the Analysis

In the following, we consider three cost measures as cost for rearranging the input using Algorithm 1. The first two cost measures aim to describe the memory behavior of Algorithm 1. The first measure counts how often each array cell is accessed during

rearranging the input, which in practice gives a good approximation on the time the CPU has to wait for memory, even when the data are in cache. We will show later that this theoretical cost measures allows us to describe practical cost measures like the average number of cache misses accurately. The second cost measure counts how often the algorithm writes into an array cell. The last cost measure is more classical and counts how many assignments the algorithm makes. It will be interesting to see that while these cost measures appear to be similar, only the first one will correctly reflect advantages of a multi-pivot quicksort approach in empirical running time. The first cost measure was also considered for the YBB algorithm in Nebel et al. [2016].

Scanned Elements. Assume that a pointer l is initialized with value l_s . Let l_e be the value in l after the algorithm finished rearranging the input. Then we define $\text{cost}(l) = |l_s - l_e|$, that is, the number of array cells inspected by pointer l . (Note that a cell is accessed only once per pointer, all pointers move by increments or decrements of 1, and $A[l_e]$ is not inspected.) Let the variable P^{sc} be the *number of scanned elements* of Algorithm 1. It is the sum of the costs of pointers $i, j, b_1, \dots, b_{k-1}$. From an empirical point of view, this cost measure gives a lower bound on the number of clock cycles the CPU spends waiting for memory. It can also be used to predict the cache behavior of Algorithm 1. We will see that it gives good estimates for the cache misses in L1 cache that we observed in our experiments. This has also been observed for the YBB algorithm in Nebel et al. [2016].

Write Accesses. Each rotate operation of ℓ elements of Algorithm 1 writes into exactly ℓ array cells. When we assign a value to an array, we call the access to this array cell a *write access*. Let the variable P^{wa} be the *number of write accesses* to array cells (over all rotate operations).

Assignments. Each rotate operation of ℓ elements of Algorithm 1 makes exactly $\ell + 1$ assignments. Let the variable P^{as} be the *number of assignments* over all rotate operations. Since each swap operation consists of three assignments, this is the most classical cost measure with respect to the three cost measures introduced above for the analysis of quicksort.

Setup of the Analysis. In the following, we want to obtain the leading term for the average number of scanned elements, write accesses, and assignments for both partitioning and over the whole sorting process. The input is again assumed to be a random permutation of the set $\{1, \dots, n\}$ that resides in an array $A[1..n]$. Fix an integer $k \geq 1$. The first k elements are chosen as pivots. Then we can think of the input consisting of $n - k$ elements having labels from $A_0, \dots, A_k$, and our goal is to rearrange the input. (In terms of multi-pivot quicksort, our goal is to obtain a partition of the input, as depicted in Figure 1. However, here determining to which of the groups $A_0, \dots, A_k$ element $A[i]$ belongs is for free.) We are interested in the cost of the rearrangement process and the total sorting cost in terms of the cost measures introduced above.

From Partitioning Cost to Sorting Cost. Let P_n denote the partitioning cost that the algorithm incurs in the first partitioning/rearrangement step. Let the random variable C_n count the sorting cost (over the whole recursion) of sorting an input of length n in the respective cost measure. As before, we get the recurrence:

$$E(C_n) = E(P_n) + \frac{1}{\binom{n}{k}} \sum_{a_0 + \dots + a_k = n-k} (E(C_{a_0}) + \dots + E(C_{a_k})).$$

Again, this recurrence has the form of Equation (2), so we may apply Equation (3) for linear partitioning cost. Thus, from now on, we focus on a single partitioning step.

7.5. Analysis

Our goal in this section is to prove the following theorem. A discussion of this result will be given in the next section.

THEOREM 7.1. *Let $k \geq 1$ be the number of pivots and $m = \lceil \frac{k+1}{2} \rceil$. Then, for Algorithm 1, we have that*

$$E(P_n^{\text{se}}) = \begin{cases} \frac{m+1}{2} \cdot n + O(1), & \text{for odd } k, \\ \frac{m^2}{2m-1} \cdot n + O(1), & \text{for even } k, \end{cases} \quad (13)$$

$$E(P_n^{\text{wa}}) = \begin{cases} \frac{2m^3 + 3m^2 - m - 2}{2m(2m+1)} \cdot n + O(1), & \text{for odd } k, \\ \frac{2m^3 - 2m - 1}{2m(2m-1)} \cdot n + O(1), & \text{for even } k, \end{cases} \quad (14)$$

$$E(P_n^{\text{as}}) = \begin{cases} \frac{2m^3 + 6m^2 - m - 4}{2m(2m+1)} \cdot n + O(1), & \text{for odd } k, \\ \frac{2m^3 + 3m^2 - 5m - 2}{2m(2m-1)} \cdot n + O(1), & \text{for even } k. \end{cases} \quad (15)$$

From this theorem, we get the leading term of the total sorting cost in the respective cost measure by applying Equation (3).

Scanned Elements. We will first study how many elements are scanned by the pointers used in Algorithm 1 when sorting an input.

Let the pivots and thus $a_0, \dots, a_k$ be fixed. The pointers i and j together scan the whole array, and thus inspect $n - k$ array cells. When Algorithm 1 terminates, b_1 points to $A[k + a_0 + 1]$, having visited exactly a_0 array cells. An analogous statement can be made for the pointers $b_2, \dots, b_{k-1}$. On average, we have $(n - k)/(k + 1)$ elements of each group $A_0, \dots, A_k$, so b_1 and b_{k-1} each visit $(n - k)/(k + 1)$ array cells on average, b_2 and b_{k-2} each visit $2(n - k)/(k + 1)$ array cells, and so on.

Consequently, for the average number of scanned elements in a partitioning step we get

$$E(P_n^{\text{se}}) = \begin{cases} 2 \cdot \sum_{i=1}^{\lceil k/2 \rceil} \frac{i \cdot (n - k)}{k + 1}, & \text{for odd } k, \\ 2 \cdot \sum_{i=1}^{\frac{k}{2}} \frac{i \cdot (n - k)}{k + 1} + \frac{k/2 + 1}{k + 1} \cdot (n - k), & \text{for even } k, \end{cases} \quad (16)$$

and a simple calculation shows

$$E(P_n^{\text{se}}) = \begin{cases} \frac{m+1}{2} \cdot (n - k), & \text{for odd } k, \\ \frac{m^2}{2m-1} \cdot (n - k), & \text{for even } k. \end{cases} \quad (17)$$

Write Accesses. We now focus on the average number of write accesses. First, we observe that a rotate operation involving ℓ elements in Algorithm 1 makes exactly ℓ element scans and ℓ write accesses. So the only difference between element scans and write accesses is that whenever pointer i finds an A_{m-1} -element in Line 7 or pointer j finds an A_m -element in Line 12, the element is scanned but no write access takes place. Let $C_{i,m-1}$ be the random variable that counts the number of A_{m-1} -elements found in Line 7, and let $C_{j,m}$ be the random variable that counts the number of A_m -elements found in Line 12 of Algorithm 1.

Thus, we know that

$$E(P_n^{\text{wa}}) = E(P_n^{\text{se}}) - E(C_{i,m-1}) - E(C_{j,m}). \quad (18)$$

LEMMA 7.2. *Let k be the number of pivots and let $m = \lceil \frac{k+1}{2} \rceil$. Then*

$$E(C_{i,m-1}) + E(C_{j,m}) = \begin{cases} \frac{m+1}{m(2m+1)} \cdot n + O(1), & \text{for } k \text{ odd,} \\ \frac{2m+1}{2m(2m-1)} \cdot n + O(1), & \text{for } k \text{ even.} \end{cases}$$

PROOF. We start by obtaining bounds on $E(C_{i,m-1})$ and $E(C_{j,m})$ when k is even. The calculations for the case when k is odd are simpler because of symmetry. In the calculations, we will consider the two events that the groups $A_0, \dots, A_{m-1}$ have L elements in total, for $0 \leq L \leq n-k$, and that group A_{m-1} has K elements, for $0 \leq K \leq L$. If the group sizes are as above, then the expected number of A_{m-1} elements scanned by pointer i is $L \cdot K/(n-k)$. We first observe that

$$\begin{aligned} E(C_{i,m-1}) &= \sum_{L=0}^n \sum_{K=0}^L \Pr(a_0 + \dots + a_{m-1} = L \wedge a_{m-1} = K) \cdot L \cdot \frac{K}{n-k} \\ &\stackrel{(*)}{=} \frac{1}{n \cdot m} \sum_{L=0}^n \Pr(a_0 + \dots + a_{m-1} = L) \cdot L^2 + O(1) \\ &= \frac{1}{n \cdot m} \sum_{L=1}^n \frac{\binom{L-1}{m-1} \binom{n-L}{m-2}}{\binom{n}{2(m-1)}} \cdot L^2 + O(1), \end{aligned} \quad (19)$$

where $(*)$ follows by noticing that $\sum_{K=0}^L \Pr(a_{m-1} = K \mid a_0 + \dots + a_{m-1} = L) \cdot K$ is the expected size of the group A_{m-1} given that the first m groups have exactly L elements, which is L/m . We calculate the sum of binomial coefficients as in Equation (19) for a more general situation:

CLAIM 7.3. *Let ℓ_1 and ℓ_2 be arbitrary integers. Then we have*

$$\begin{aligned} \text{(i)} \quad & \sum_{L=1}^n \frac{\binom{L-1}{\ell_1} \binom{n-L}{\ell_2}}{\binom{n}{\ell_1+\ell_2+1}} \cdot L^2 = \frac{(\ell_1+1)(\ell_1+2) \cdot (n+2)(n+1)}{(\ell_1+\ell_2+2)(\ell_1+\ell_2+3)} - \frac{(\ell_1+1) \cdot (n+1)}{(\ell_1+\ell_2+2)}. \\ \text{(ii)} \quad & \sum_{L=1}^n \frac{\binom{L-1}{\ell_1} \binom{n-L}{\ell_2}}{\binom{n}{\ell_1+\ell_2+1}} \cdot (n-L)^2 = \frac{(\ell_2+1)(\ell_2+2) \cdot (n+2)(n+1)}{(\ell_1+\ell_2+2)(\ell_1+\ell_2+3)} - \frac{3(\ell_2+1) \cdot (n+1)}{\ell_1+\ell_2+2} + 1. \end{aligned}$$

PROOF. We denote by n^k the k th falling factorial of n , that is, $n(n-1)\cdots(n-k+1)$. Using known identities for sums of binomial coefficients, we may calculate

$$\begin{aligned}
& \sum_{L=1}^n \frac{\binom{L-1}{\ell_1} \binom{n-L}{\ell_2}}{\binom{n}{\ell_1+\ell_2+1}} \cdot L^2 \\
&= \frac{1}{\binom{n}{\ell_1+\ell_2+1}} \sum_{L=1}^n \left((\ell_1+1)(\ell_1+2) \binom{L+1}{\ell_1+2} \binom{n-L}{\ell_2} - L \binom{L-1}{\ell_1} \binom{n-L}{\ell_2} \right) \\
&= \frac{1}{\binom{n}{\ell_1+\ell_2+1}} \sum_{L=1}^n \left((\ell_1+1)(\ell_1+2) \binom{L+1}{\ell_1+2} \binom{n-L}{\ell_2} - (\ell_1+1) \binom{L}{\ell_1+1} \binom{n-L}{\ell_2} \right) \\
&\stackrel{(*)}{=} \frac{1}{\binom{n}{\ell_1+\ell_2+1}} \left((\ell_1+1)(\ell_1+2) \binom{n+2}{\ell_1+\ell_2+3} - (\ell_1+1) \binom{n+1}{\ell_1+\ell_2+2} \right) \\
&= \frac{(\ell_1+1)(\ell_1+2)(n+2) \frac{\ell_1+\ell_2+3}{(\ell_1+\ell_2+3)!} (\ell_1+\ell_2+1)!}{(\ell_1+\ell_2+3)! \cdot n^{\ell_1+\ell_2+1}} - \frac{(\ell_1+1)(n+1) \frac{\ell_1+\ell_2+2}{(\ell_1+\ell_2+2)!} (\ell_1+\ell_2+1)!}{(\ell_1+\ell_2+2)! \cdot n^{\ell_1+\ell_2+1}} \\
&= \frac{(\ell_1+1)(\ell_1+2)(n+2)(n+1)}{(\ell_1+\ell_2+2)(\ell_1+\ell_2+3)} - \frac{(\ell_1+1)(n+1)}{(\ell_1+\ell_2+2)},
\end{aligned}$$

where $(*)$ follows by using the identity [Graham et al. 1994, (5.26)]. The calculations for (ii) are analogous by using an index transformation $K = n - L$. $\square$

Using the claim, we continue from Equation (19) as follows:

$$E(C_{i,m-1}) = \frac{m(m+1) \cdot (n+1)(n+2)}{nm \cdot (2m-1)2m} + O(1) = \frac{m+1}{2m(2m-1)}n + O(1). \quad (20)$$

By similar arguments, we obtain

$$\begin{aligned}
E(C_{j,m}) &= \frac{1}{n \cdot (m-1)} \sum_{L=0}^n \frac{\binom{L}{m-1} \binom{n-L}{m-2}}{\binom{n}{2(m-1)}} \cdot (n-L)^2 + O(1) \\
&= \frac{(m-1)m}{2nm(m-1)(2m-1)}n^2 + O(1) = \frac{1}{2(2m-1)}n + O(1). \quad (21)
\end{aligned}$$

Thus, in the asymmetric case it holds that $E(C_{i,m-1}) + E(C_{j,m}) = \frac{2m+1}{2m(2m-1)}n + O(1)$.

Applying Lemma 7.2 to Equations (13) and (18) and simplifying gives us the values from Theorem 7.1, Equation (14).

Assignments. To count the total number of assignments, we first observe that each rotate operation that involves ℓ elements makes ℓ write accesses and $\ell+1$ assignments. Thus, the total number of assignment is the sum of the number of write accesses and the number of rotate operations. So we observe

$$E(P_n^{\text{as}}) = E(P_n^{\text{wa}}) + E(\# \text{rotate operations}). \quad (22)$$

LEMMA 7.4. *Let k be the number of pivots and $m = \lceil \frac{k+1}{2} \rceil$. Then it holds that*

$$E(\# \text{rotate operations}) = \begin{cases} \frac{3m^2-2}{2m(2m+1)} + O(1), & \text{for odd } k, \\ \frac{3m^2-3m-1}{2m(2m-1)} + O(1), & \text{for even } k. \end{cases}$$

PROOF. The number of rotate operations is counted as follows. For each non- A_{m-1} element that is scanned by pointer i , a rotate operation is invoked (Line 9 and Line 18

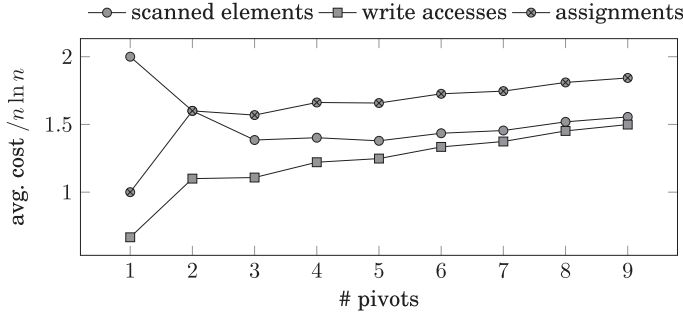


Fig. 8. Average number of scanned elements, write accesses, and assignments for sorting an input of length n using Algorithm 1.

in Algorithm 1). In addition, each $A_{m'}$ element with $m' > m$ scanned by pointer j invokes a rotate operation (Line 14 in Algorithm 1). So the number of rotate operations is the sum of these two quantities. Again, we focus on the case that k is even. Let $C_{i,<m-1}$ be the number of $A_{m'}$ elements with $m' < m-1$ scanned by pointer i . Define $C_{i,>m-1}$ and $C_{j,>m}$ analogously. By symmetry (cf. Equation (19)) we have that

$$E(C_{i,<m-1}) = (m-1) \cdot E(C_{i,m-1}) = \frac{(m-1)(m+1)}{2m(2m-1)}n + O(1),$$

see Equation (20). Furthermore, since we expect that pointer i scans $\frac{m}{2m-1}(n-k)$ elements, we know that

$$E(C_{i,>m-1}) = \frac{m}{2m-1}(n-k) - m \cdot E(C_{i,m-1}) = \left(\frac{m}{2m-1} - \frac{m+1}{2(2m-1)} \right)n + O(1).$$

Finally, again by symmetry, we obtain

$$E(C_{j,>m}) = (m-2) \cdot E(C_{j,m}) = \frac{m-2}{2(2m-1)}n + O(1).$$

For even k the result now follows by adding these three values. For odd k , we only have to adjust the formulas to the fact that we expect that pointer i scans $(n-k)/2$ elements and that there are $m-1$ groups $A_{m+1}, \dots, A_k$ when calculating $E(C_{j,>m})$. $\square$

Applying Lemma 7.4 to Equation (22) and simplifying gives the values from Theorem 7.1, Equation (15).

7.6. Discussion and Empirical Validation

Using the formulae developed in the previous subsection, we calculated the average number of scanned elements, write accesses, and assignments in partitioning and in sorting for $k \in \{1, \dots, 9\}$ using Theorem 7.1 and Equation (3). Figure 8 gives a graphical overview of these calculations. Next, we will discuss our findings.

Interestingly, Algorithm 1 improves over classical quicksort when using more than one pivot with regard to scanned elements. A three-pivot quicksort algorithm, using this partitioning algorithm, has lower cost ($1.385n \ln n$) than classical ($2n \ln n$) and dual-pivot quicksort ($1.6n \ln n$). Moreover, the average number of scanned elements is minimized by the five-pivot partitioning algorithm ($1.379n \ln n$ scanned elements on average). However, the difference to the three-pivot algorithm is small. Using more than five pivots increases the average number of scanned elements. With respect to write accesses and assignments, we see a different picture. In both cost measures, the average sorting cost increases considerably when moving from classical quicksort to

Table II. Cache Misses Incurred by Algorithm 1 (“Exchange_k”) in a Single Partitioning Step.
All Values Are Averaged Over 600 Trials

Algorithm	Exchange ₁	Exchange ₂	Exchange ₅	Exchange ₉
avg. L1 misses / n	0.125	0.163	0.25	0.378

quicksort variants with at least two pivots. For a growing number of pivots, it slowly increases further. In conclusion, Algorithm 1 benefits from using more than one pivot only with respect to scanned elements but not with respect to the average number of write accesses and assignments.

In Aumüller [2015, Chapter 7], two very different partitioning algorithms are considered. They are based on the super scalar sample sort algorithm of Sanders and Winkel [2004], which is an implementation of samplesort [Frazer and McKellar 1970]. These algorithms classify the elements in a first pass and use these classifications to move the elements efficiently around in a second pass. While these approaches lead to higher cost for a small number of pivots, they theoretically outperform the generalized partitioning algorithm considered here for, for example, 127 pivots.

Running Time Implications. We now ask what the considerations made so far mean for empirical running time. Since each memory access, even if it can be served from L1 cache, is much more expensive than other operations, like simple subtraction, addition, or assignments on or between registers, the results for the cost measure “scanned elements” show that for different multi-pivot quicksort algorithms we must expect big differences in the time the CPU has to wait for memory.⁶ If, in addition, writing an element back into cache/memory is more expensive than reading from the cache (as it could happen with the “write-through” cache strategy), then the calculations show that we should not expect advantages of multi-pivot quicksort algorithms over classical quicksort in terms of memory behavior. However, cache architectures in modern CPUs apply the “write-back” strategy, which does not add a penalty to running time for writing into memory.

As is well known from classical quicksort and dual-pivot quicksort, the influence of lower-order terms cannot be neglected for real-world values of n . Next, we will validate our findings for practical values of n .

Empirical Validation. We implemented Algorithm 1 and ran it for different input lengths and pivot numbers. In the experiments, we sorted inputs of size 2^i with $9 \leq i \leq 27$. Each data point is the average over 600 trials. For measuring cache misses, we used the “performance application programming interface,” which is available at <http://icl.cs.utk.edu/papi/>.

Intuitively, fewer scanned elements should yield better cache behavior when memory accesses are done “scanlike” as in the algorithms considered here. The argument used in LaMarca and Ladner [1999] and Kushagra et al. [2014] is as follows: When each of the m cache memory blocks holds exactly B keys, then a scan of n' array cells (that have never been accessed before) incurs $\lceil n'/B \rceil$ cache misses. Now we check whether the assertion that partitioning an input of n elements using Algorithm 1 incurs $\lceil E(P_n^{\text{se}})/B \rceil$ cache misses is justifiable. (Recall that $E(P_n^{\text{se}})$ is the average number of scanned elements during partitioning.) In the experiment, we partitioned 600 inputs consisting of $n = 2^{27}$ items using Algorithm 1, for 1, 2, 5, and 9 pivots. The measurements with respect to L1 cache misses are shown in Table II. In our setup, each L1 cache line contains eight

⁶As an example, the Intel i7 used in our experiments needs at least four clock cycles to read from memory if the data is in L1 cache and its physical address is known. If the data are in L2 cache but not in L1 cache, then there is an additional penalty of 6 clock cycles. On the other hand, three (data-independent) MOV operations between registers on the same core take only 1 clock cycle. See Fog [2014] for more details.

Table III. Average Number of L1/L2 Cache Misses Compared to the Average Number of Scanned Elements for Sorting Inputs of Size $n = 2^{27}$. Cache Misses Are Scaled by $n \ln n$ and Are Averaged Over 600 Trials. In Parentheses, We Show the Ratio to the Best Algorithmic Variant of Algorithm 1 W.R.T. Memory/Cache Behavior ($k = 5$), Calculated from the Non-Truncated Experimental Data

Algorithm	$E(C_n^{\text{se}})$	L1 Cache Misses	L2 Cache Misses
Exchange ₁	$2.000n \ln n (+ 45.0\%)$	$0.140n \ln n (+ 48.9\%)$	$0.0241n \ln n (+263.1\%)$
Exchange ₂	$1.600n \ln n (+ 16.0\%)$	$0.110n \ln n (+ 16.9\%)$	$0.0124n \ln n (+ 86.8\%)$
Exchange ₃	$1.385n \ln n (+ 0.4\%)$	$0.096n \ln n (+ 1.3\%)$	$0.0080n \ln n (+ 19.8\%)$
Exchange ₅	$1.379n \ln n (-)$	$0.095n \ln n (-)$	$0.0067n \ln n (-)$
Exchange ₇	$1.455n \ln n (+ 5.5\%)$	$0.100n \ln n (+ 5.3\%)$	$0.0067n \ln n (+ 0.7\%)$
Exchange ₉	$1.555n \ln n (+ 12.8\%)$	$0.106n \ln n (+ 12.2\%)$	$0.0075n \ln n (+ 12.9\%)$

elements. So, by Equation (13) Algorithm 1 should theoretically incur $0.125n$, $0.166n$, $0.25n$, and $0.375n$ L1 cache misses for $k \in \{1, 2, 5, 9\}$, respectively. The results from Table II show that the empirical measurements are very close to these values.

Table III shows the exact measurements regarding L1 and L2 cache misses for sorting 600 random inputs consisting of $n = 2^{27}$ elements using Algorithm 1 and relates them to each other.⁷ The figures indicate that the relation with respect to the measured number of L1 cache misses of the different algorithms reflect their relation with respect to the average number of scanned elements *very well*. However, while the average number of cache misses correctly reflects the relative relations, the measured values (scaled by $n \ln n$) are lower than we would expect by simply dividing $E(C_n^{\text{se}})$ by the block size B . We give the following heuristic argument that adjusts the theoretical formulas to this effect. First, it is no surprise that the values are too high since the recursion to compute the sorting cost changes if true cache misses are counted. Once the whole input is contained in the L1 cache, no cache misses are involved. So, the actual cost in terms of cache misses for sorting these inputs is 0. Let the average number of scanned elements during partitioning be $an + O(1)$. Then we adjust our estimate on the average number of L1 cache misses by subtracting $a/(B(\mathcal{H}_{k+1} - 1))n \ln M$ from our estimate $E(C_n^{\text{se}})/B$ for a constant M that depends on the actual machine.⁸ Numerical computations for the measured number of L1 cache misses showed that setting $M = 3751$, which is very close to the number of cache lines in the L1 cache of the machine used measuring cache misses, gives results very close to the measured number of L1 cache misses. We remark that this approach does not give good estimates for L2 cache misses.

In summary, scanned elements appear to be a suitable cost measure to predict the L1 cache behavior of Algorithm 1. However, this is not true with regard to L2 cache behavior of these algorithms, as shown in Table III.

Figure 9 shows the measurements we got with regard to the average number of assignments. We see that the measurements agree with our theoretical study (cf. Figure 8). In particular, lower-order terms seem to have a low influence on the sorting cost.

Conclusion. In this section, we analyzed an algorithm (Algorithm 1) for the rearrangement problem with respect to three different cost measures. We found that the

⁷We omit our measurements for L3 cache misses. In contrast to our measurements for L1 and L2 cache misses, the measurements on this level of the hierarchy differed substantially in each run. We believe this is due to L3 caches being shared among cores.

⁸It is well known for classical quicksort and dual-pivot quicksort [Wild et al. 2015] that changing the recurrence to charge cost $P_{n'} = 0$ for sorting inputs of length n' , for $n' \leq M$, influences only the linear term in the average sorting cost. For example, Wild et al. [2015] showed that for the dual-pivot quicksort recurrence the solution for average partitioning cost $an + O(n^{1-\epsilon})$ is $(6/5)an \ln(n/(M+2)) + O(n)$. We did not check the exact influence of stopping the recursion for the multipivot recursion at these input sizes but assume that it behaves similarly.

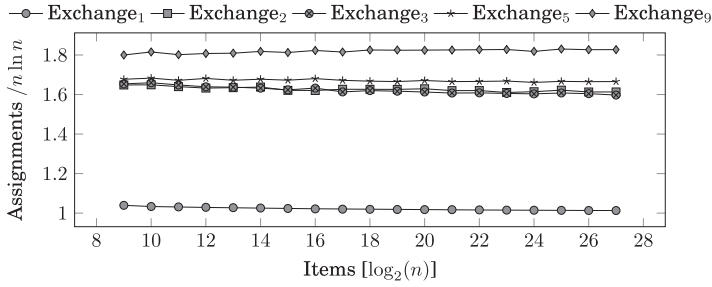


Fig. 9. The average number of assignments for sorting a random input consisting of n elements using Algorithm 1 (“Exchange _{k} ”) for certain values of k . Each data point is the average over 600 trials.

cost measure “scanned elements” is very useful for estimating the number of cache misses. With respect to the number of scanned elements, Algorithm 1 is particularly good with three to five pivots. For the cost measures “write accesses” and “assignments,” we found that the cost increases with an increasing number of pivots.

8. PIVOT SAMPLING IN MULTI-PIVOT QUICKSORT

In this section, we study possible benefits of sampling pivots with respect to comparisons and to scanned elements. By “pivot sampling,” we mean that we take a sample of elements from the input, sort these elements, and then pick certain elements of this sorted sequence as pivots. One particularly popular strategy for classical quicksort, known as *median-of-three*, is to choose as a pivot the median of a sample of three elements [van Emde 1970]. From a theoretical point of view, it is well known that choosing the median in a sample of $\Theta(\sqrt{n})$ elements in classical quicksort is optimal with respect to minimizing the average comparison count [Martínez and Roura 2001]. Using this sample size, quicksort achieves the (asymptotically) best possible average comparison count of $(1/\ln 2)n \ln n = 1.4426 \dots n \ln n + O(n)$ comparisons on average. For the YBB algorithm, Nebel, Wild, and Martínez [2016] studied the influence of pivot sampling in detail. In particular, they considered the setting where one can choose pivots of a given rank for free. (Of course, this only gives a lower bound on the sorting cost.) In this model, they proved that no matter how well the pivots are chosen, the YBB algorithm makes at least $1.49 \dots n \ln n + O(n)$ comparisons on average. Aumüller and Dietzfelbinger [2015] demonstrated that this is not an inherent limitation of the dual-pivot quicksort approach.

Here we study two different sampling scenarios for multi-pivot quicksort. First, we develop formulae to calculate the average number of comparisons and the average number of scanned elements for Algorithm 1 when pivots are chosen from a small sample. Example calculations demonstrate that the cost in both measures can be decreased by choosing pivots from a small (fixed-sized) sample. Interestingly, with respect to scanned elements, the best pivot choices do not balance subproblem sizes but tend to make the middle groups, that is, groups A_p with p close to $\lceil \frac{k+1}{2} \rceil$, larger. Then we consider the different setting in which we can choose pivots of a given rank for free. For this setting, we wish to find out which pivot choices minimize the respective cost. Our first result shows that if we choose an arbitrary comparison tree and use it in every classification, it is possible to choose pivots in such a way that, on average, we need at most $1.4426 \dots n \ln n + O(n)$ comparisons to sort the input, which is optimal. In a second result, we identify a particular pivot choice that minimizes the average number of scanned elements. In contrast to the results of the previous section, we show that with these pivot choices, the average number of scanned elements decreases with a growing number of pivots. (Recall that when choosing pivots directly from the input, the five-pivot

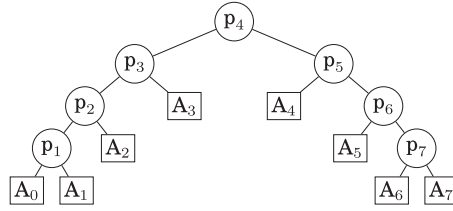


Fig. 10. The extremal tree for seven pivots.

quicksort algorithm based on Algorithm 1 has minimal cost.) From these calculations, we also learn which comparison tree (among the exponentially many available) has the lowest cost when considering as a cost measure the sum of the number of comparisons and the number of scanned elements. In contrast to intuition, the balanced comparison tree, in which all leaves are as even in depth as possible, has non-optimal cost. The best choice under this cost measure is to use the comparison tree that uses as root pivot p_m with $m = \lceil \frac{k+1}{2} \rceil$. In its left subtree, the node labeled with pivot p_i is the left child of the node labeled with pivot p_{i+1} for $1 \leq i \leq m-1$. (So the inner nodes in its left subtree are a path $(p_{m-1}, \dots, p_1)$.) Analogously, in its right subtree, the node labeled with pivot p_{i+1} is the right child of the node labeled with pivot p_i for $m \leq i \leq k-1$. For given $k \geq 1$, we call this tree the *extremal tree for k pivots*. In Figure 10, we see an example for the extremal tree for seven pivots.

General Structure of a Multi-Pivot Quicksort Algorithm Using Sampling. We generalize a multi-pivot quicksort algorithm in the following way. (This description is analogous to those in Hennequin [1991] for multi-pivot quicksort and Nebel et al. [2016] for dual-pivot quicksort.) For a given number $k \geq 1$ of pivots, we fix a vector $\mathbf{t} = (t_0, \dots, t_k) \in \mathbb{N}^{k+1}$. Let $\kappa := \kappa(\mathbf{t}) = k + \sum_{0 \leq i \leq k} t_i$ be the number of samples.⁹ Assume that an input of n elements residing in an array $A[1..n]$ is to be sorted. If $n \leq \kappa$, then sort A directly. Otherwise, sort the first κ elements and then set $p_i = A[i + \sum_{j < i} t_j]$ for $1 \leq i \leq k$. Next, partition the input $A[\kappa + 1..n]$ with respect to the pivots $p_1, \dots, p_k$. Subsequently, by a constant number of rotations, move the elements residing in $A[1..\kappa]$ to correct final locations. Finally, sort the $k+1$ subproblems recursively.

The sampling technique described above does not preserve randomness in subproblems, because some elements have already been sorted during the pivot sampling step. For the analysis, we ignore that the unused samples have been seen and get only an estimate on the sorting cost. A detailed analysis of this situation for dual-pivot quicksort is given in Nebel et al. [2016]; the same methods that were proposed in their article for a tighter analysis can be used in the multi-pivot quicksort case, too.

The Generalized Multi-Pivot Quicksort Recurrence. For a given sequence $\mathbf{t} = (t_0, \dots, t_k) \in \mathbb{N}^{k+1}$, we define $H(\mathbf{t})$ by

$$H(\mathbf{t}) = \sum_{i=0}^k \frac{t_i + 1}{\kappa + 1} (H_{\kappa+1} - H_{t_i+1}). \quad (23)$$

Let P_n denote the random variable that counts the cost of a single partitioning step, and let C_n denote the cost over the whole sorting procedure. In general, we get the

⁹The notation differs from Hennequin [1991] and Nebel et al. [2016] in the following way: This article focuses on a “pivot number”-centric approach, where the main parameter is k , the number of pivots. The other articles focus on the parameter s , the number of element groups. In particular, it holds $s = k + 1$. The sample size κ is denoted k in these articles.

recurrence

$$E(C_n) = E(P_n) + \sum_{a_0 + \dots + a_k = n-k} (E(C_{a_0}) + \dots + E(C_{a_k})) \cdot \Pr(\langle a_0, \dots, a_k \rangle), \quad (24)$$

where $\langle a_0, \dots, a_k \rangle$ is the event that the group sizes are exactly $a_0, \dots, a_k$. The probability of this event for a given vector $\mathbf{t}$ is

$$\frac{\binom{a_0}{t_0} \dots \binom{a_k}{t_k}}{\binom{n}{\kappa}}.$$

For the following discussion, we re-use the result of Hennequin [1991, Proposition III.9], which says that for fixed k and $\mathbf{t}$ and average partitioning cost $E(P_n) = a \cdot n + O(1)$ recurrence (24) has the solution

$$E(C_n) = \frac{a}{H(\mathbf{t})} n \ln n + O(n). \quad (25)$$

The Average Comparison Count Using a Fixed Comparison Tree. Fix a vector $\mathbf{t} \in \mathbb{N}^{k+1}$. First, observe that for each $i \in \{0, \dots, k\}$, the expected number of elements belonging to group A_i is $\frac{t_i+1}{\kappa+1}(n-\kappa)$. If the $n-\kappa$ remaining input elements are classified using a fixed comparison tree λ , then the average comparison count for partitioning (cf. Equation (5)) is

$$(n-\kappa) \cdot \sum_{i=0}^k \text{depth}_\lambda(A_i) \cdot \frac{t_i+1}{\kappa+1}. \quad (26)$$

The Average Number of Scanned Elements of Algorithm 1. Fix a vector $\mathbf{t} \in \mathbb{N}^{k+1}$ and let $m = \lceil \frac{k+1}{2} \rceil$. Arguments analogous to the ones presented in the previous section, see Equation (16), show that the average number of scanned elements of Algorithm 1 is

$$\begin{cases} n \cdot \sum_{i=1}^m i \cdot \left(\frac{t_{m-i} + t_{k-m+i} + 2}{\kappa+1} \right) + O(1), & \text{for } k \text{ odd,} \\ n \cdot (m+1) \cdot \frac{t_0+1}{\kappa+1} + n \cdot \sum_{i=1}^m i \cdot \left(\frac{t_{m+1-i} + t_{k-m+i} + 2}{\kappa+1} \right) + O(1), & \text{for } k \text{ even.} \end{cases} \quad (27)$$

Next, we will use these formulae to give some example calculations for small sample sizes.

Optimal Pivot Choices for Small Sample Sizes. Table IV contains the lowest possible cost for a given number of pivots and a given number of sample elements. We consider three different cost measures: the average number of comparisons, the average number of scanned elements, and the sum of these two costs. Additionally, Table IV contains the $\mathbf{t}$ -vector and the comparison tree that achieves this value. (Of course, each comparison tree yields the same number of scanned elements. Consequently, no comparison tree is given for the best $\mathbf{t}$ -vector w.r.t. scanned elements.)

Looking at Table IV, we make the following observations. Increasing the sample size for a fixed number of pivots decreases the average cost significantly, at least asymptotically. Interestingly, to minimize the average number of comparisons, the best comparison tree is not always one that minimizes the depth of the tree, for example, see the best comparison tree for five pivots and six additional samples. However, for most situations, the tree with minimal cost has minimal depth. To minimize the number of scanned elements, the groups in the middle should be made larger. The extremal tree provides the best possible *total cost*, summing up comparisons and scanned elements. Compared to the sampling choices that minimize the number of scanned elements, the

Table IV. Best Sampling and Comparison Tree Choices for a Given Number k of Pivots and a Given Sample Size (in Addition to Pivots). A Comparison Tree Is Presented as the Output of the Preorder Traversal of Its Inner Nodes, for example, the Extremal Tree in Figure 10 Is Described By [4, 3, 2, 1, 5, 6, 7]. Since the Number of Scanned Elements Is Independent of the Used Comparison Tree, No Comparison Tree Is Given for This Cost Measure

k	Add. Samples	Cost measure	Best candidate (cost, t , tree)
3	0	comparisons	$1.846n \ln n$, (0, 0, 0, 0), [2, 1, 3]
		scanned elements	$1.385n \ln n$, (0, 0, 0, 0), —
		cmp + scanned elements	$3.231n \ln n$, (0, 0, 0, 0), [2, 1, 3]
	4	comparisons	$1.642n \ln n$, (1, 1, 1, 1), [2, 1, 3]
		scanned elements	$1.144n \ln n$, (0, 2, 2, 0), —
		cmp + scanned elements	$2.874n \ln n$, (1, 1, 1, 1), [2, 1, 3]
	8	comparisons	$1.575n \ln n$, (2, 2, 2, 2), [2, 1, 3]
		scanned elements	$1.098n \ln n$, (1, 3, 3, 1), —
		cmp + scanned elements	$2.745n \ln n$, (1, 3, 3, 1), [2, 1, 3]
	16	comparisons	$1.522n \ln n$, (4, 4, 4, 4), [2, 1, 3]
		scanned elements	$1.055n \ln n$, (2, 6, 6, 2), —
		cmp + scanned elements	$2.627n \ln n$, (3, 5, 5, 3), [2, 1, 3]
5	0	comparisons	$1.839n \ln n$, (0, 0, 0, 0, 0, 0), [3, 2, 1, 4, 5]
		scanned elements	$1.379n \ln n$, (0, 0, 0, 0, 0, 0), —
		cmp + scanned elements	$3.218n \ln n$, (0, 0, 0, 0, 0, 0), [3, 2, 1, 4, 5]
	6	comparisons	$1.635n \ln n$, (0, 0, 0, 2, 2, 2), [4, 3, 1, 2, 5]
		scanned elements	$1.097n \ln n$, (0, 1, 2, 2, 1, 0), —
		cmp + scanned elements	$2.741n \ln n$, (0, 1, 2, 2, 1, 0), [3, 2, 1, 4, 5]
	12	comparisons	$1.567n \ln n$, (1, 1, 4, 4, 1, 1), [3, 2, 1, 4, 5]
		scanned elements	$1.019n \ln n$, (0, 1, 5, 5, 1, 0), —
		cmp + scanned elements	$2.635n \ln n$, (1, 1, 4, 4, 1, 1), [3, 2, 1, 4, 5]
	16	comparisons	$1.746n \ln n$, (0, 0, 0, 0, 0, 0, 0), [4, 2, 1, 3, 6, 5, 7]
		scanned elements	$1.455n \ln n$, (0, 0, 0, 0, 0, 0, 0), —
		cmp + scanned elements	$3.201n \ln n$, (0, 0, 0, 0, 0, 0, 0), [4, 2, 1, 3, 6, 5, 7]
		comparisons	$1.595n \ln n$, (1, 1, 1, 1, 1, 1, 1), [4, 2, 1, 3, 6, 5, 7]
		scanned elements	$1.094n \ln n$, (0, 0, 1, 3, 3, 1, 0, 0), —
		cmp + scanned elements	$2.698n \ln n$, (0, 0, 1, 3, 3, 1, 0, 0), [4, 3, 2, 1, 5, 6, 7]
7	0	comparisons	$1.544n \ln n$, (2, 2, 2, 2, 2, 2, 2), [4, 2, 1, 3, 6, 5, 7]
		scanned elements	$1.017n \ln n$, (0, 0, 2, 6, 6, 2, 0, 0), —
		cmp + scanned elements	$2.594n \ln n$, (0, 0, 2, 6, 6, 2, 0, 0), [4, 3, 2, 1, 5, 6, 7]
	8	comparisons	$1.763n \ln n$, (0, 0, 0, 0, 0, 0, 0, 0), [5, 3, 2, 1, 4, 7, 6, 8, 9]
		scanned elements	$1.555n \ln n$, (0, 0, 0, 0, 0, 0, 0, 0), —
		cmp + scanned elements	$3.318n \ln n$, (0, 0, 0, 0, 0, 0, 0, 0), [5, 3, 2, 1, 4, 7, 6, 8, 9]
	10	comparisons	$1.602n \ln n$, (0, 0, 1, 2, 2, 2, 2, 1, 0, 0), [5, 3, 2, 1, 4, 7, 6, 8, 9]
		scanned elements	$1.131n \ln n$, (0, 0, 0, 1, 4, 4, 1, 0, 0, 0), —
		cmp + scanned elements	$2.748n \ln n$, (0, 0, 0, 1, 4, 4, 1, 0, 0, 0), [5, 4, 3, 2, 1, 6, 7, 8, 9]
	20	comparisons	$1.543n \ln n$, (1, 1, 2, 3, 3, 3, 3, 2, 1, 1), [5, 3, 2, 1, 4, 7, 6, 8, 9]
		scanned elements	$1.040n \ln n$, (0, 0, 0, 2, 8, 8, 2, 0, 0, 0), —
		cmp + scanned elements	$2.601n \ln n$, (0, 0, 1, 2, 7, 7, 2, 1, 0, 0), [5, 4, 3, 2, 1, 6, 7, 8, 9]

sampled elements are slightly less concentrated around the middle element groups. This is first evidence that the extremal tree might be the best possible comparison tree for a given number of pivots with respect to the total cost.

With regard to the question of the best choice of k , Table IV shows that it is not possible to give a definite answer. All of the considered pivot numbers and additional sampling elements make it possible to decrease the total average sorting cost to around $2.6n \ln n$ using only a small sample.

Next, we will study the behavior of these cost measures when choosing the pivots is for free.

Optimal Pivot Choices. We now consider the following setting. We assume that for a random input of n elements¹⁰ we can choose (for free) k pivots w.r.t. a vector $\tau = (\tau_0, \dots, \tau_k)$ such that the input contains exactly $\tau_i n$ elements from group A_i , for $i \in \{0, \dots, k\}$. By definition, we have $\sum_{0 \leq i \leq k} \tau_i = 1$. This setting was studied in Martínez and Roura [2001] and Nebel et al. [2016] as well.

We make the following preliminary observations. In our setting, the average number of comparisons *per element* (see Equation (26)) becomes

$$c_\tau := \sum_{i=0}^k \text{depth}_t(A_i) \cdot \tau_i, \quad (28)$$

and the average number of scanned elements per element (see Equation (27)) is

$$a_\tau := \begin{cases} \sum_{i=1}^{m-1} i \cdot (\tau_{m-i-1} + \tau_{k-m+i+1}), & \text{for } k \text{ odd,} \\ m \cdot \tau_0 + \sum_{i=1}^{m-1} i \cdot (\tau_{m-i} + \tau_{k-m+i+1}), & \text{for } k \text{ even.} \end{cases} \quad (29)$$

Furthermore, using that the κ th harmonic number H_κ is approximately $\ln \kappa$, we get that $H(\tau)$ from Equation (23) converges to the entropy of τ , which is

$$\mathcal{H}(\tau) := - \sum_{i=0}^k \tau_i \ln \tau_i.$$

In the following we want to obtain optimal choices for the vector τ to minimize the three values

$$\frac{c_\tau}{\mathcal{H}(\tau)}, \quad \frac{a_\tau}{\mathcal{H}(\tau)}, \quad \frac{c_\tau + a_\tau}{\mathcal{H}(\tau)}, \quad (30)$$

that is, the values that minimize the factor in the $n \ln n$ term of the sorting cost in the respective cost measure.

We start by giving a general solution to the problem of minimizing functions f over the simplex $S_{k+1} = \{(\tau_0, \dots, \tau_k) \in \mathbb{R}^{k+1} \mid \tau_0, \dots, \tau_k \geq 0, \sum_i \tau_i = 1\}$, where f is the quotient of some linear function in $\tau_0, \dots, \tau_k$ and the entropy of $\tau_0, \dots, \tau_k$, as in Equation (30).

LEMMA 8.1. *Let $\alpha_0, \dots, \alpha_k > 0$ be arbitrary constants. For $\tau = (\tau_0, \dots, \tau_k) \in S_{k+1}$ define*

$$f(\tau) = \frac{\alpha_0 \tau_0 + \dots + \alpha_k \tau_k}{\mathcal{H}(\tau)}.$$

Let x be the unique solution in $(0, 1)$ of the equation

$$1 = x^{\alpha_0} + x^{\alpha_1} + \dots + x^{\alpha_k}.$$

¹⁰We disregard the k pivots in the following discussion.

Then $\tau = (x^{\alpha_0}, x^{\alpha_1}, \dots, x^{\alpha_k})$ minimizes $f(\tau)$ over S_{k+1} , that is, $f(\tau) \leq f(\sigma)$ for all $\sigma \in S_{k+1}$. The minimum value is $-1/\ln x$.

PROOF. Gibb's inequality says that for arbitrary nonnegative $q_0, \dots, q_k$ with $\sum_i q_i \leq 1$ and arbitrary nonnegative $p_0, \dots, p_k$ with $\sum_i p_i = 1$, the following holds:

$$\mathcal{H}(p_0, \dots, p_k) \leq \sum_{i=0}^k p_i \ln\left(\frac{1}{q_i}\right).$$

Now consider $x \in (0, 1)$ such that $\sum_i x^{\alpha_i} = 1$ and set $\tau_i = x^{\alpha_i}$. By Gibb's inequality, for arbitrary $(\sigma_0, \dots, \sigma_k) \in S_{k+1}$ we have

$$\mathcal{H}(\sigma_0, \dots, \sigma_k) \leq \sum_{i=0}^k \sigma_i \ln\left(\frac{1}{x^{\alpha_i}}\right) = \sum_{i=0}^k \sigma_i \alpha_i \ln\left(\frac{1}{x}\right),$$

hence

$$f(\sigma_0, \dots, \sigma_k) = \frac{\sum_{i=0}^k \sigma_i \alpha_i}{\mathcal{H}(\sigma_0, \dots, \sigma_k)} \geq \frac{1}{\ln\left(\frac{1}{x}\right)}.$$

Finally, observe that

$$f(\tau_0, \dots, \tau_k) = \frac{\sum_{i=0}^k \tau_i \alpha_i}{\mathcal{H}(\tau_0, \dots, \tau_k)} = \frac{\sum_{i=0}^k \alpha_i x^{\alpha_i}}{-\sum_{i=0}^k x^{\alpha_i} \ln(x^{\alpha_i})} = \frac{1}{\ln\left(\frac{1}{x}\right)}. \quad \square$$

We first consider optimal choices for the sampling vector τ in order to minimize the average number of comparisons. The following theorem says that each comparison tree makes it possible to achieve the minimum possible sorting cost for comparison-based sorting algorithms in the considered setting.

THEOREM 8.2. *Let $k \geq 1$ be fixed. Let $\lambda \in \Lambda^k$ be an arbitrary comparison tree. Then there exists τ such that the average comparison count using comparison tree λ in each classification is $(1/\ln 2)n \ln n + O(n) = 1.4426..n \ln n + O(n)$.*

PROOF. For each $i \in \{0, \dots, k\}$, set $\tau_i = 2^{-\text{depth}_\lambda(A_i)}$. First, observe that $\sum \tau_i = 1$. (This is true since every inner node in λ has exactly two children.) Starting from Equation (26), we may calculate:

$$\sum_{i=0}^k \text{depth}_\lambda(A_i) \cdot \tau_i = \sum_{i=0}^k -\log(\tau_i) \cdot \tau_i = -\frac{1}{\ln 2} \sum_{i=0}^k \tau_i \cdot \ln(\tau_i).$$

This shows the theorem. $\square$

We now consider the average number of scanned elements of Algorithm 1. The following theorem says that pivots should be chosen in such a way that (in the limit for $k \rightarrow \infty$) $2/3$ of the input are only scanned once (by the pointers i and j), $2/9$ should be scanned twice, and so on.

THEOREM 8.3. *Let $k \geq 1$ be fixed. Let $m = \lceil \frac{k+1}{2} \rceil$. Let τ be chosen according to the following two cases:*

(1) *If k is odd, then let x be the unique value in $(0, 1)$ such that*

$$1 = 2(x + x^2 + \dots + x^m).$$

$$\text{Let } \tau = (x^m, x^{m-1}, \dots, x, x, \dots, x^m).$$

(2) If k is even, then let x be the unique value in $(0, 1)$ such that

$$1 = 2(x + x^2 + \dots + x^{m-1}) + x^m.$$

Let $\tau = (x^m, x^{m-1}, \dots, x, x, \dots, x^{m-1})$.

Then the average number of scanned elements using Algorithm 1 with τ is minimal over all choices of vectors τ' . For $k \rightarrow \infty$, this minimum is $(1/\ln 3)n \ln n \approx 0.91n \ln n$ scanned elements.

PROOF. Setting the values $\alpha_0, \dots, \alpha_k$ in Lemma 8.1 according to Equation (29) shows that the choices for τ are optimal with respect to minimizing the average number of scanned elements. One easily checks that in the limit for $k \rightarrow \infty$ the value x in the statement is $1/3$. $\square$

For example, for the YBB algorithm, Nebel et al. [2016] noticed that $\tau = (q^2, q, q)$ with $q = \sqrt{2} - 1$ is the optimal pivot choice to minimize element scans. In this case, around $1.13n \ln n$ elements are scanned on average. The minimal average number of scanned elements using Algorithm 1 for $k \in \{3, 5, 7, 9\}$ are around $0.995n \ln n$, $0.933n \ln n$, $0.917n \ln n$, and $0.912n \ln n$, respectively. Hence, already for small values of k the average number of scanned elements is close to $0.91n \ln n$. However, from Table IV, we see that for sample sizes suitable in practice, both the average comparison count and average pointer visit count are around $0.1n \ln n$ higher than these asymptotic values.

To summarize, we learned that (i) every fixed comparison tree yields an optimal classification strategy and (ii) one specific pivot choice has the best possible average number of scanned elements in Algorithm 1. Next, we consider as cost measure the sum of the average number of comparisons and the average number of scanned elements.

THEOREM 8.4. Let $k \geq 1$ be fixed. Let $m = \lceil \frac{k+1}{2} \rceil$. Let τ be chosen according to the following two cases:

(1) If k is odd, then let x be the unique value in $(0, 1)$ such that

$$1 = 2(x^3 + x^5 + \dots + x^{2m-3} + x^{2m-1} + x^{2m}).$$

Let $\tau = (x^{2m}, x^{2m-1}, x^{2m-3}, \dots, x^3, x^3, \dots, x^{2m-3}, x^{2m-1}, x^{2m})$.

(2) If k is even, then let x be the unique value in $(0, 1)$ such that

$$1 = 2(x^3 + x^5 + \dots + x^{2m-3}) + x^{2m-2} + x^{2m-1} + x^{2m}.$$

Let $\tau = (x^{2m}, x^{2m-1}, x^{2m-3}, \dots, x^3, x^3, \dots, x^{2m-3}, x^{2m-2})$.

Then the average cost using Algorithm 1 and classifying all elements with the extremal comparison tree for k pivots using τ is minimal among all choices of vectors τ' . For $k \rightarrow \infty$ this minimum cost is about $2.38n \ln n$.

PROOF. First, observe that for the extremal comparison tree for k pivots, Equation (28) becomes

$$c_\tau = \begin{cases} m(\tau_0 + \tau_k) + \sum_{i=1}^{m-1} (i+1)(\tau_{m-i} + \tau_{m+i-1}), & \text{for } k \text{ odd,} \\ m(\tau_0 + \tau_1) + (m-1)\tau_k + \sum_{i=1}^{m-2} (i+1)(\tau_{m-i} + \tau_{m+i-1}), & \text{for } k \text{ even.} \end{cases} \quad (31)$$

The optimality of the choice for τ in the theorem now follows from adding Equation (29) to Equation (31) and using Lemma 8.1. For $k \rightarrow \infty$, the optimal x value is the unique solution in $(0, 1)$ of the equation $2x^3 + x^2 = 1$, which is about 0.6573. Thus, the total cost is about $2.38n \ln n$. $\square$

Interestingly, the minimal total cost of $2.38n \ln n$ is only about $0.03n \ln n$ higher than adding $(1/\ln 2)n \ln n$ (the minimal average comparison count) and $0.91n \ln n$ (the minimal average number of scanned elements). Using the extremal tree is much better than, for example, using the balanced comparison tree for $k = 2^k - 1$ pivots. The minimum sorting cost using this tree is $2.489n \ln n$, which is achieved for three pivots.¹¹ We conjecture that the extremal tree has minimal total sorting cost, that is, minimizes the sum of scanned elements and comparisons, but must leave this as an open problem. We checked this conjecture via exhaustive search for $k \leq 9$ pivots.

Again, including scanned elements as a cost measure yields unexpected results and design recommendations for engineering a sorting algorithm. Looking at comparisons, the balanced tree for $2^k - 1$ pivots is the obvious comparison tree to use in a multi-pivot quicksort algorithm. Only when including scanned elements does the extremal tree shows its potential in leading to fast multi-pivot quicksort algorithms.¹²

9. EXPERIMENTS

In this section, we address the question of how the design recommendations of the theoretical findings in this article work out in practice. We stress that we tested our algorithms strictly in the theoretical setting of random permutations as studied throughout this article. So, our algorithms are far from “library-ready” implementations that work nicely on different input distributions, especially in the presence of equal keys.

Objectives of the Experiments. The key theoretical findings of the article suggest the following questions about running times of multi-pivot quicksort algorithms.

- Q1 *Can comparison-optimal multi-pivot quicksort compete in running time?* While the (asymptotically) comparison-optimal multi-pivot quicksort algorithms from Section 5 minimize the average comparison count, they add a time overhead to obtain the optimal comparison tree in each classification step. In this light, it seems that they cannot compete with simpler algorithms that use a fixed tree.
- Q2 *How do running times of multi-pivot quicksort algorithms compare?* In Section 7, we pointed out that using three to five pivots in Algorithm 1 yields the best memory behavior. We study how this is reflected in running times.
- Q3 *Is the extremal comparison tree faster than the balanced comparison tree?* In Section 8, we showed that in a certain cost model multi-pivot quicksort algorithms using extremal classification trees have lower cost than balanced comparison trees. We investigate whether this corresponds to a difference in running times or not.
- Q4 *Is pivot sampling beneficial for running time?* For a fixed sample vector τ , choosing the pivots in a skewed way as proposed in Theorem 8.3 has lower cost than using equidistant pivots (Section 8) and greatly improves over the case where pivots are chosen directly from the input. We examine whether such sampling choices yield noticeable differences with regard to running time.

Experimental Setup. We implemented all algorithms in C++ and used *clang* in version 3.5 for compiling with the optimization flag *-O3*. Our experiments were carried out on an Intel i7-2600 at 3.4GHz with 16GB RAM running Ubuntu 14.10 with kernel version 3.16.0.

We restricted our experiments to sorting random permutations of the integers $\{1, \dots, n\}$. We tested inputs of size 2^i , $21 \leq i \leq 27$. For each input size, we ran each algorithm on the same 600 random permutations. All figures in the following are the

¹¹For three pivots, the extremal tree and the balanced tree are identical.

¹²It is interesting to note that articles dealing with multi-pivot quicksort such as Hennequin [1991], Kushagra et al. [2014], and Iliopoulos [2014] do not consider the full design space for partitioning strategies but rather always assume the “most-balanced” tree is best.

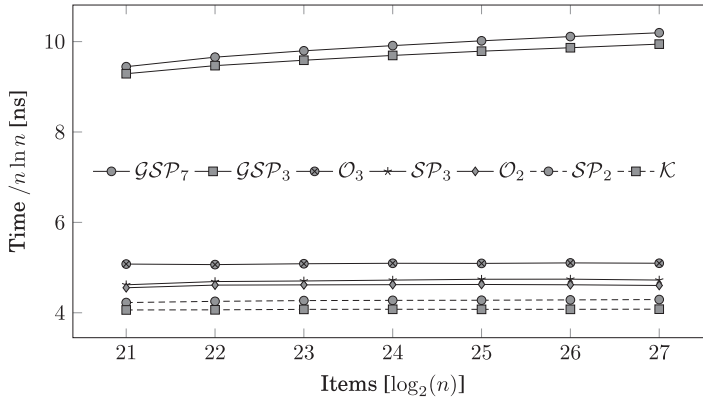


Fig. 11. Running time experiments for sorting inputs of size 2^i for $21 \leq i \leq 27$. The algorithms shown are three- (GSP_3) and seven-pivot (GSP_7) algorithms that compute the optimal comparison tree and use the sampling approach, two- and three-pivot variants of the optimal algorithm O_k and its sampling variant SP_k that choose the optimal comparison tree directly, and the three-pivot algorithm K of Kushagra et al. [2014] as described in Aumüller and Dietzfelbinger [2015]. Each data point is the average over 600 trials. Times are scaled by $n \ln n$.

average over these 600 trials. We say that a running time improvement is *significant* if it was observed in at least 95% of the trials.

To deal with Questions 2 and 4, we wrote a script to generate multi-pivot quicksort algorithms with a fixed pivot choice based on Algorithm 1. (We remark that manually writing the algorithm code is error-prone and tedious. For example, the source code for the nine-pivot algorithm uses 376 lines of code and needs nine different rotate operations.) In these algorithms, subarrays of size at most 500 were sorted using our implementation of the fast three-pivot quicksort algorithm of Kushagra et al. [2014] as described in Aumüller and Dietzfelbinger [2015]. The source code of all algorithms and the code generator is available at <http://eiche.theoinf.tu-ilmenau.de/quicksort-experiments/>.

At the end of this section, we compare the results to the standard introsort sorting method from C++’s standard library and the super scalar sample sort (“SSSS”) algorithm of Sanders and Winkel [2004].

9.1. Experimental Evaluation Regarding Questions 1–4

Question 1: Can comparison-optimal multi-pivot quicksort compete in running time?

We implemented the comparison-optimal algorithm O_k and the (asymptotically) optimal sample strategy SP_k for two and three pivots such that the optimal comparison tree was chosen directly by comparing group sizes. For example, in the three-pivot algorithm, the optimal comparison tree of the five possible comparison trees is chosen by selecting the tree that has the smallest cost term in Equation (9). In addition, we implemented k -pivot variants that choose the optimal comparison tree algorithmically. We computed the tree using Knuth’s dynamic programming approach [Knuth 1973]. (We did not implement the algorithm of Garsia and Wachs.) The time measurements of our experiments with comparison-optimal multi-pivot quicksort algorithms are shown in Figure 11. We used the implementation of the three-pivot algorithm K from Kushagra et al. [2014] as described in Aumüller and Dietzfelbinger [2015] as the baseline to which to compare the algorithms. This three-pivot algorithm was the fastest algorithm, being around 5% faster on average than its closest competitor SP_2 . We see that the sampling variant SP_k is faster than the comparison-optimal variant O_k (by about 7% both for two and three pivots). The two-pivot sampling algorithm is about 9.2% faster than its

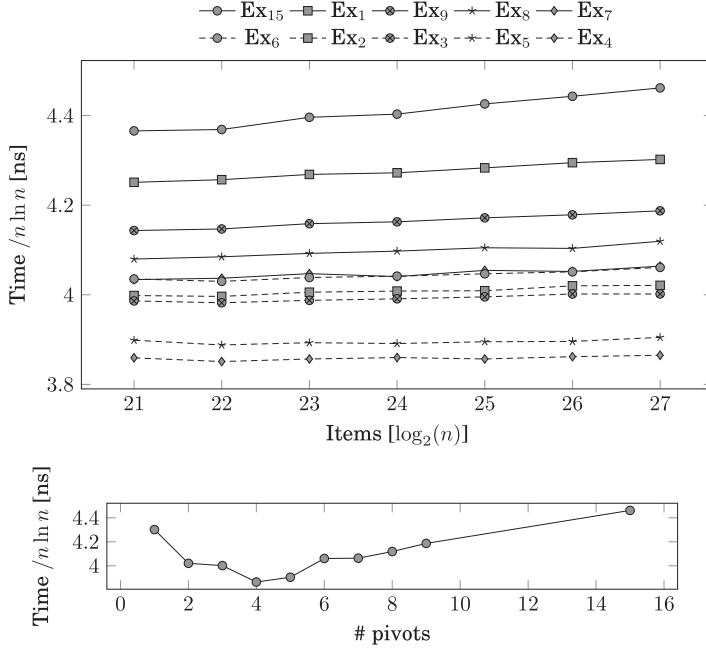


Fig. 12. Running-time experiments for k -pivot quicksort algorithms based on the “Exchange _{k} ” partitioning strategy. Each data point is the average over 600 trials. Times are scaled by $n \ln n$. Top: Running times for sorting inputs of size 2^i for $21 \leq i \leq 27$. Bottom: Running times from top with respect to the number of pivots used for sorting inputs of size 2^{27} .

three-pivot variant. The generic sampling variants that choose the tree algorithmically are far slower than these algorithms. The generic three-pivot sampling implementation $\mathcal{GSP}_3$ is about 2.2 times slower than its direct counterpart $\mathcal{SP}_3$. On average, the generic seven-pivot sampling implementation $\mathcal{GSP}_7$ is only about 3% slower than $\mathcal{GSP}_3$. We omit the running time for the generic comparison-optimal algorithms. For three pivots, it was about 5 times slower on average than its sampling variant $\mathcal{GSP}_3$. We conclude that comparison-optimal multi-pivot quicksort cannot compete in running time when sorting random permutations of integers, especially when the comparison tree is chosen algorithmically.

Question 2: How do running times of multi-pivot quicksort algorithms compare? The time measurements of our experiments based on k -pivot quicksort algorithms generated automatically from Algorithm 1 are shown in Figure 12. With respect to average running time, we see that the variants using four and five pivots are fastest. On average the 5-pivot algorithm is about 1% slower than the 4-pivot algorithm. However, there is no significant difference in running time between these two variants. On average, the 3-pivot and 2-pivot algorithm are 3.5% and 3.7% slower than the 4-pivot quicksort algorithm. The 6- and 7-pivot algorithms are about 5.0% slower. It follows the 8-pivot algorithm (6.5% slower), the 9-pivot algorithm (8.5% slower), classical quicksort (11.0% slower), and the 15-pivot algorithm (15.5% slower). If we only consider significant differences in running time, then these figures must be decreased by about 1–2%. These results are in line with our study of scanned elements in Section 7; see in particular the second column of Table III. This shows that the cost measure “scanned elements” describes observed running time differences very well. For a larger number of pivots, additional work, for example, finding and sorting the pivots, seems to have a noticeable

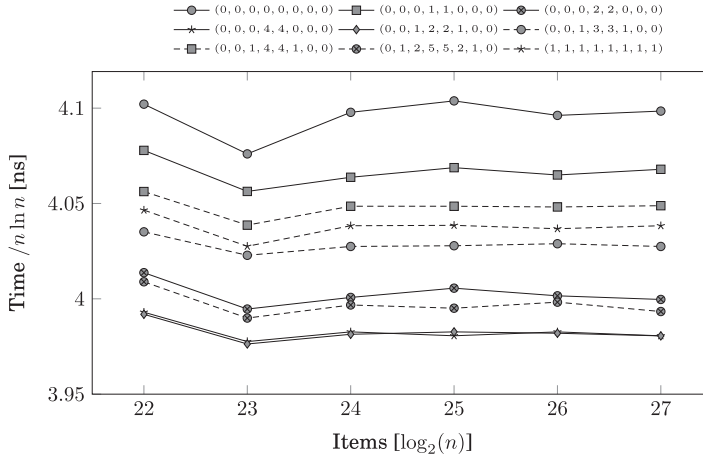


Fig. 13. Running-time experiments for different sampling strategies for the seven-pivot quicksort algorithm based on the “Exchange₇” partitioning strategy. Each line represents the running-time measurements obtained with the given τ sampling vector. Each data point is the average over 600 trials. Times are scaled by $n \ln n$.

influence on running time. For example, according to the average number of scanned elements the 15-pivot quicksort should not be slower than classical quicksort.

Question 3: Is the extremal comparison tree faster than the balanced comparison tree? We report on an experiment that compared a seven-pivot algorithm using the extremal comparison tree and a seven-pivot algorithm using the balanced comparison tree. For inputs of size $n = 2^{27}$, the seven-pivot quicksort algorithm with the extremal tree was at least 2% faster than the seven-pivot algorithm with the balanced tree in 95% of the runs. The difference in running time is thus not large but statistically significant.

Question 4: Is pivot sampling beneficial for running time? Before reporting on the results of our experiments, we want to stress that comparing the overhead incurred by sorting a larger sample to the benefits of having better pivots is very difficult from a theoretical point of view because of the influence of lower-order terms to the sorting cost. In these experiments, we observed that sampling strategies that make the groups closer to the center larger improved the running time more significantly than balanced samples, which validates our findings from Section 8. However, the benefits regarding empirical running time observed when pivots are chosen from a small sample are minimal. Compared to choosing pivots directly from the input, the largest improvements in running time were observed for the seven- and nine-pivot algorithms. For these algorithms, the running time could be improved by about 3% by choosing pivots from a sample of size 13 or 15. For example, Figure 13 depicts the running times we obtained for variants using seven pivots and different sampling vectors. In some cases, for example, for the four-pivot algorithm, we could not observe any improvements by sampling pivots.¹³

¹³Note that sampling improves the cache behavior. For example, the four-pivot algorithm without sampling incurred 10% more L1 cache misses, 10% more L2 cache misses, and needed 7% more instructions than the four-pivot algorithm with sampling vector (0, 0, 1, 1, 0). Still, it was 2% faster in experiments. The reason might be that it made 7% less branch mispredictions.

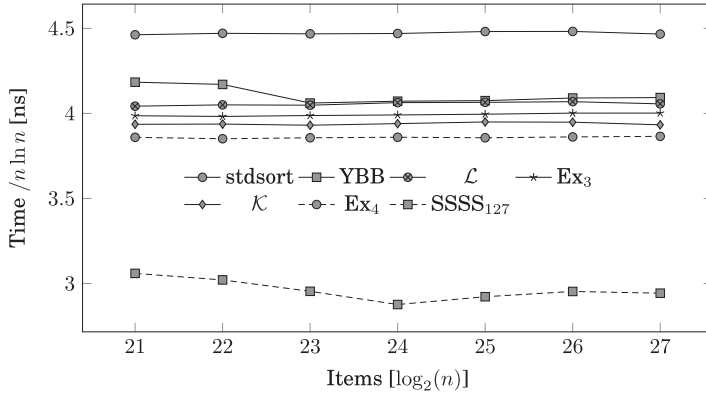


Fig. 14. Running-time experiments for different quicksort-based algorithms compared to the “Exchange_k” partitioning strategies Ex₃ and Ex₄. The plot includes the average running time of C++’s `std::sort` implementation (“stdsort”), the YBB algorithm (“YBB”), the dual-pivot quicksort algorithm from Aumüller and Dietzfelbinger [2015, Algorithm 3] (“ $\mathcal{L}$ ”), the three-pivot quicksort algorithm from Kushagra et al. (“ $\mathcal{K}$ ”), and the super-scalar sample sort variant of Sanders and Winkel [2004] using 127 pivots (“SSSS₁₂₇”). Each data point is the average over 600 trials. Times are scaled by $n \ln n$.

9.2. Comparison with Other Methods

Here, we report on experiments in which the algorithms from before were compared with other quicksort-based algorithms known from the literature. For the comparison, we used the `std::sort` implementation found in C++’s standard STL (from gcc), the YBB algorithm from Nebel et al. [2016, Figure 4], the two-pivot algorithm from Aumüller and Dietzfelbinger [2015, Algorithm 3], the three-pivot algorithm of Kushagra et al. [2014], see Aumüller and Dietzfelbinger [2015, Algorithm 8], and an implementation of the super scalar sample sort algorithm of Sanders and Winkel [2004] with basic source code for classifications provided by Timo Bingmann. Algorithm `std::sort` is an introsort implementation, which combines quicksort with a heap-sort fallback when subproblem sizes decrease too slowly. As explained in Sanders and Winkel [2004], using a large number of pivots in a sample sort approach can be implemented to exploit *data independence* (classifications are decoupled from each other) and *predicated move instructions*, which reduce branch mispredictions in the classification step. See the source code at <http://eiche.theoinf.tu-ilmenau.de/quicksort-experiments/> for details.

Figure 14 shows the measurements we got for these algorithms.¹⁴ We see that `std::sort` is by far the slowest algorithm. Of course, it is slowed down by special precautions for inputs that are not random or have equal entries. (Such precautions have not been taken in the other implementations.) Next come the two dual-pivot quicksort algorithms. The three-pivot algorithm of Kushagra et al. [2014] is a little bit faster than the automatically generated three-pivot quicksort algorithm but a little slower than the automatically generated four-pivot quicksort algorithm. (This shows that the automatically generated quicksort algorithms do not suffer in performance compared to manually tuned implementations such as the three-pivot algorithm from Kushagra et al. [2014].) The implementation of the super scalar sample sort algorithm using 127 pivots of Sanders and Winkel is by far the fastest algorithms. However, it is only faster if classifications are stored in the first pass and no re-classifications take

¹⁴We used gcc version 4.9 to compile the YBB algorithm and the super scalar sample sort algorithm. For both algorithms, the executable obtained by compiling with clang was much slower. For the YBB algorithm, compiling with the compiler flag `-O3` was fastest, while for SSSS flags `-O3 -funroll-loops` was fastest.

place in the second pass. Using 255 pivots is slightly slower, and using 511 is much slower.

In conclusion, the experiments validated our theoretical results from before. It is interesting to see that the memory-related cost measure “scanned elements” comes closest to explain running times. It seems unlikely that algorithms based on the partitioning method from Algorithm 1 provide drastic improvements in running time when used with more than five pivots. For sorting random permutations, the benefits of pivot sampling seem only minimal. When a lot of space and good compiler/architecture support are available, drastic improvements in running time are possible using two-pass quicksort variants such as the super scalar sample sort algorithm of Sanders and Winkel [2004]. During the process of reviewing this article, a variant of a single-pivot quicksort algorithm with running time comparable to super scalar sample sort but without additional memory has been proposed by Edelkamp and Weiss [2016]. As in super scalar sample sort, the running-time improvement comes from a trick to avoid branches.

10. CONCLUSION

In this article, we studied the design space of multi-pivot quicksort algorithms and demonstrated how to analyze their sorting cost. In the first part, we showed how to calculate the average comparison count of an arbitrary multi-pivot quicksort algorithm. We described optimal multi-pivot quicksort algorithms with regard to key comparisons. It turned out that calculating their average comparison count seems difficult already for four pivots (we resorted to experiments) and that they offer no drastic improvement with respect to comparisons with respect to other methods. Similar improvements in key comparisons can be achieved by much simpler strategies such as the median-of- k strategy for classical quicksort. From a running-time point of view, comparison-optimal k -pivot quicksort is not competitive in running time to simpler quicksort variants that, for example, use some fixed comparison tree, even when we bypass the computation of an optimal comparison tree. In the second part, we switched our viewpoint and studied the problem of rearranging entries to obtain a partition of the input. We analyzed a one-pass algorithm (Algorithm 1) with respect to the cost measures “scanned elements,” “write accesses,” and “assignments.” For the second and third cost measure, we found that the cost increases with an increasing number of pivots. The first cost measure turned out to be more interesting. Using Algorithm 1 with three or five pivots was particularly good, and we asserted that “scanned elements” correspond to L1 cache misses in practice. Experiments revealed that there is a high correlation to observed running times. In the last part of this article, we discussed the influence of pivot sampling to sorting cost. With respect to comparisons we showed that for every comparison tree we can describe a pivot choice such that the cost is optimal. With regard to the number of scanned elements, we noticed that pivots should be chosen such that they make groups closer to the center larger. To determine element groups, the extremal comparison tree should be used. We conjectured that this choice of tree is also best when we consider as cost the sum of comparisons and scanned elements.

For future work, it would be very interesting to see how the optimal average comparison count of k -pivot quicksort can be calculated analytically, cf. Equation (12). The situation seems to be much more complicated than for dual-pivot quicksort, for which Aumüller et al. [2016] were able to obtain the *exact* average comparison count in very recent work. With respect to the rearrangement problem, it would be interesting to identify an optimal algorithm that shifts as few elements as possible to rearrange an input. With respect to pivot sampling, we conjecture that the extremal tree minimizes the sorting cost when counting comparisons and scanned elements. This is true for up to nine pivots, but it remains open to prove it for general values of k . From an empirical

point of view, it would be interesting to test multi-pivot quicksort algorithms on input distributions that are not random permutations but have some kind of “presortedness” or contain equal keys. Very recent work [Edelkamp and Weiss 2016] showed that our variants perform worse on inputs containing equal elements than other standard approaches. How to deal with these elements efficiently should be investigated in a separate study on the topic. Initial experiments for two and three pivots showed that running times greatly improve when we make sure that whenever two pivots are the same, we put all elements equal to these pivots between them and avoid the recursive call. However, there are many ways of doing this and it would be very interesting to see what turns out to be most efficient both in theory and in practice. Again with respect to running times, a key technique in algorithm engineering is to avoid branch mispredictions. Techniques that do this have been successfully applied in super scalar sample sort [Sanders and Winkel 2004], tuned quicksort [Elmasry et al. 2012], and the recent “Block Quicksort” algorithm [Edelkamp and Weiss 2016]. It would be interesting to see how one can apply these techniques for multi-pivot quicksort for a small number of two to five pivots.

APPENDIX

A. SOLVING THE GENERAL QUICKSORT RECURRENCE

In Section 2, we have shown that the sorting cost of k -pivot quicksort satisfies the following recurrence:

$$E(C_n) = E(P_n) + \frac{1}{\binom{n}{k}} \sum_{i=0}^{n-k} (k+1) \binom{n-i-1}{k-1} E(C_i).$$

We will use the continuous Master theorem (CMT) of Roura [2001] to solve this recurrence. For completeness, we give the CMT below:

THEOREM A.1 ([MARTÍNEZ AND ROURA 2001, THEOREM 18]). *Let F_n be recursively defined by*

$$F_n = \begin{cases} b_n, & \text{for } 0 \leq n < N, \\ t_n + \sum_{j=0}^{n-1} w_{n,j} F_j, & \text{for } n \geq N, \end{cases}$$

where the toll function t_n satisfies $t_n \sim Kn^\alpha \log^\beta(n)$ as $n \rightarrow \infty$ for constants $K \neq 0, \alpha \geq 0, \beta > -1$. Assume there exists a function $w : [0, 1] \rightarrow \mathbb{R}$ such that

$$\sum_{j=0}^{n-1} \left| w_{n,j} - \int_{j/n}^{(j+1)/n} w(z) dz \right| = O(n^{-d}), \quad (32)$$

for a constant $d > 0$. Let $H := 1 - \int_0^1 z^\alpha w(z) dz$. Then we have the following cases:¹⁵

- (1) If $H > 0$, then $F_n \sim t_n/H$.
- (2) If $H = 0$, then $F_n \sim (t_n \ln n)/\hat{H}$, where

$$\hat{H} := -(\beta + 1) \int_0^1 z^\alpha \ln(z) w(z) dz.$$

¹⁵Let $f(n)$ and $g(n)$ be two functions. We write $f(n) \sim g(n)$ if $f(n) = g(n) + o(g(n))$. If $f(n) \sim g(n)$, then we say that “ f and g are asymptotically equivalent.”

(3) If $H < 0$, then $F_n \sim \Theta(n^c)$ for the unique $c \in \mathbb{R}$ with

$$\int_0^1 z^c w(z) dz = 1.$$

THEOREM A.2. *Let A be a k -pivot quicksort algorithm that for each subarray of length n has partitioning cost $E(P_n) = a \cdot n + O(n^{1-\varepsilon})$. Then*

$$E(C_n) = \frac{1}{H_{k+1} - 1} a n \ln n + O(n),$$

where $H_{k+1} = \sum_{i=1}^{k+1} (1/i)$ is the $(k+1)$ st harmonic number.

PROOF. By linearity of expectation, we may obtain a solution for the recurrence for toll function $t_{1,n} = a \cdot n$ and toll function $t_{2,n} = K \cdot n^{1-\varepsilon}$ separately and add the solutions.

For toll function $t_{1,n}$, we use the result of Hennequin [1991, Proposition III.9] that says that for partitioning cost $a \cdot n + O(1)$ we get sorting cost $E(C_{1,n}) = \frac{a}{H_{k+1}-1} n \ln n + O(n)$.

For $t_{2,n}$, we apply the CMT as follows. First, observe that Recurrence (2) has weight

$$w_{n,j} = \frac{(k+1) \cdot k \cdot (n-j-1) \cdot \dots \cdot (n-j-k+1)}{n \cdot (n-1) \cdot \dots \cdot (n-k+1)}.$$

We define the shape function $w(z)$ as suggested in Roura [2001] by

$$w(z) = \lim_{n \rightarrow \infty} n \cdot w_{n,zn} = (k+1)k(1-z)^{k-1}.$$

Using the Binomial theorem for asymptotic bounds, we note that for all $z \in [0, 1]$:

$$\begin{aligned} |n \cdot w_{n,zn} - w(z)| &\leq k \cdot (k+1) \cdot \left| \frac{(n-zn-1)^{k-1}}{(n-k)^{k-1}} - (1-z)^{k-1} \right| \\ &= k \cdot (k+1) \cdot \left| \left(\frac{n(1-z)}{n-k} - \frac{1}{n-k} \right)^{k-1} - (1-z)^{k-1} \right| \\ &\leq k \cdot (k+1) \cdot \left| \left(\frac{n(1-z)}{n-k} \right)^{k-1} - (1-z)^{k-1} + O(n^{-1}) \right| \\ &\leq k \cdot (k+1) \cdot \left| (1-z)^{k-1} \cdot \left(\frac{1}{(1-\frac{k}{n})^{k-1}} - 1 \right) + O(n^{-1}) \right| \\ &\leq k \cdot (k+1) \cdot \left| (1-z)^{k-1} \cdot \left(\frac{1}{1-O(n^{-1})} - 1 \right) + O(n^{-1}) \right| \\ &\leq k \cdot (k+1) \cdot |(1-z)^{k-1} \cdot O(n^{-1}) + O(n^{-1})| = O(n^{-1}). \end{aligned}$$

Now we have to check Equation (32) to see whether the shape function is suitable. We calculate:

$$\begin{aligned}
& \sum_{j=0}^{n-1} \left| w_{n,j} - \int_{j/n}^{(j+1)/n} w(z) \, dz \right| \\
&= \sum_{j=0}^{n-1} \left| \int_{j/n}^{(j+1)/n} n \cdot w_{n,j} - w(z) \, dz \right| \\
&\leq \sum_{j=0}^{n-1} \frac{1}{n} \max_{z \in [j/n, (j+1)/n]} |n \cdot w_{n,j} - w(z)| \\
&\leq \sum_{j=0}^{n-1} \frac{1}{n} \left(\max_{z \in [j/n, (j+1)/n]} |w(j/n) - w(z)| + O(n^{-1}) \right) \\
&\leq \sum_{j=0}^{n-1} \frac{1}{n} \left(\max_{|z-z'| \leq 1/n} |w(z) - w(z')| + O(n^{-1}) \right) \\
&\leq \sum_{j=0}^{n-1} \frac{k(k+1)}{n} \left(\max_{|z-z'| \leq 1/n} |(1-z)^{k-1} - (1-z-1/n)^{k-1}| + O(n^{-1}) \right) \\
&\leq \sum_{j=0}^{n-1} O(n^{-2}) = O(n^{-1}),
\end{aligned}$$

where we again used the Binomial theorem in the last two lines.

Thus, w is a suitable shape function. Using partial integration, we see that

$$H := 1 - k(k+1) \int_0^1 z^{1-\varepsilon} (1-z)^{k-1} \, dz < 0.$$

Thus, the third case of the CMT applies. Again using partial integration, we check that

$$k(k+1) \int_0^1 z(1-z)^{k-1} \, dz = 1,$$

so we conclude that for toll function $t_{2,n}$ the recurrence has solution $E(C_{2,n}) = O(n)$. The theorem follows by adding $E(C_{1,n})$ and $E(C_{2,n})$. $\square$

ACKNOWLEDGMENTS

We thank Sebastian Wild for many interesting discussions and his valuable comments on an earlier version of this manuscript. In particular, the general partitioning algorithm from Section 7 evolved from discussions with him at Dagstuhl Seminar 14091. We also thank Conrado Martínez and Markus Nebel for interesting discussions and comments on multi-pivot quicksort. In addition, we thank Timo Bingmann for providing source code. We thank the referees of this submission for their insightful comments, which helped a lot in improving the presentation and focusing on the main insights.

REFERENCES

- Martin Aumüller and Martin Dietzfelbinger. 2013. Optimal partitioning for dual pivot quicksort. In *Proceedings of the 40th International Colloquium on Automata, Languages and Programming (ICALP'13)*. Springer, 33–44.
- Martin Aumüller and Martin Dietzfelbinger. 2015. Optimal partitioning for dual-pivot quicksort. *ACM Trans. Algor.* 12, 2 (2015), 18. DOI: <http://dx.doi.org/10.1145/2743020>

- Martin Aumüller, Martin Dietzfelbinger, Clemens Heuberger, Daniel Krenn, and Helmut Prodingner. 2016. Counting zeros in random walks on the integers and analysis of optimal dual-pivot quicksort. *CoRR* abs/1602.04031 (2016), In *Proceedings of the 27th International Conference on Probabilistic, Combinatorial and Asymptotic Methods for the Analysis of Algorithms (AofA'16)*, 3.
- Martin Aumüller. 2015. *On the Analysis of Two Fundamental Randomized Algorithms: Multi-Pivot Quicksort and Efficient Hash Functions*. Ph.D. Dissertation. Technische Universität Ilmenau.
- Jon L. Bentley and M. Douglas McIlroy. 1993. Engineering a sort function. *Softw. Pract. Exp.* 23, 11, 1249–1265.
- Jon Louis Bentley and Robert Sedgewick. 1997. Fast algorithms for sorting and searching strings. In *Proceedings of the 8th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA'97)*. ACM, 360–369.
- Joshua Bloch. 2015. Personal Communication.
- Edsger W. Dijkstra. 1976. *A Discipline of Programming*. Prentice-Hall.
- Devdatt P. Dubhashi and Alessandro Panconesi. 2009. *Concentration of Measure for the Analysis of Randomized Algorithms*. Cambridge University Press.
- Stefan Edelkamp and Armin Weiss. 2016. BlockQuicksort: Avoiding branch mispredictions in quicksort. In *Proceedings of the 24th Annual European Symposium on Algorithms (ESA'16)*. 38:1–38:16. DOI: <http://dx.doi.org/10.4230/LIPIcs.ESA.2016.38>
- Amr Elmasry, Jyrki Katajainen, and Max Stenmark. 2012. Branch mispredictions don't affect mergesort. In *Proceedings of the 11th International Symposium on Experimental Algorithms (SEA'12)*. 160–171. DOI: http://dx.doi.org/10.1007/978-3-642-30850-5_15
- Agner Fog. 2014. 4. Instruction tables. Retrieved from http://www.agner.org/optimize/instruction_tables.pdf.
- W. D. Frazer and A. C. McKellar. 1970. Samplesort: A sampling approach to minimal storage tree sorting. *J. ACM* 17, 3, 496–507. DOI: <http://dx.doi.org/10.1145/321592.321600>
- Adriano M. Garsia and Michelle L. Wachs. 1977. A new algorithm for minimum cost binary trees. *SIAM J. Comput.* 6, 4, 622–642. DOI: <http://dx.doi.org/10.1137/0206045>
- Ronald L. Graham, Donald E. Knuth, and Oren Patashnik. 1994. *Concrete Mathematics—A Foundation for Computer Science*, 2nd ed. Addison-Wesley.
- Pascal Hennequin. 1991. *Analyse en Moyenne D'algorithmes: Tri Rapide et Arbres de Recherche*. Ph.D. Dissertation. Ecole Polytechnique, Palaiseau.
- C. A. R. Hoare. 1962. Quicksort. *Comput. J.* 5, 1 (1962), 10–15.
- T. C. Hu and A. C. Tucker. 1971. Optimal computer search trees and variable-length alphabetical codes. *SIAM J. Appl. Math.* 21, 4, 514–532.
- Vasileios Iliopoulos. 2014. A note on multipivot quicksort. *CoRR* abs/1407.7459.
- Kanella Kaligosi and Peter Sanders. 2006. How branch mispredictions affect quicksort. In *Proceedings of the 14th Annual European Symposium on Algorithms (ESA'06)*. Springer, 780–791.
- Donald E. Knuth. 1973. *The Art of Computer Programming, Volume III: Sorting and Searching*. Addison-Wesley.
- Shrinu Kushagra, Alejandro López-Ortiz, Aurick Qiao, and J. Ian Munro. 2014. Multi-pivot quicksort: Theory and experiments. In *Proceedings of the 16th Meeting on Algorithms Engineering and Experiments (ALENEX'14)*. SIAM, 47–60.
- Anthony LaMarca and Richard E. Ladner. 1999. The influence of caches on the performance of sorting. *J. Algor.* 31, 1 (1999), 66–104.
- Nikolaj Leischner, Vitaly Osipov, and Peter Sanders. 2010. GPU sample sort. In *Proceedings of the 24th IEEE International Symposium on Parallel and Distributed Processing, IPDPS 2010, Atlanta, Georgia, USA, 19–23 April 2010*. 1–10. DOI: <http://dx.doi.org/10.1109/IPDPS.2010.5470444>
- David Levinthal. 2009. Performance Analysis Guide for Intel Core i7 Processor and Intel Xeon 5500 processors. Retrieved from https://software.intel.com/sites/products/collateral/hpc/vtune/performance_analysis_guide.pdf.
- Conrado Martínez, Markus E. Nebel, and Sebastian Wild. 2015. Analysis of branch misses in quicksort. In *Proceedings of the 12th Meeting on Analytic Algorithmics and Combinatorics (ANALCO'15)*. 114–128.
- Conrado Martínez and Salvador Roura. 2001. Optimal sampling strategies in quicksort and quickselect. *SIAM J. Comput.* 31, 3 (2001), 683–705.
- Peter M. McIlroy, Keith Bostic, and M. Douglas McIlroy. 1993. Engineering radix sort. *Comput. Syst.* 6, 1 (1993), 5–27.
- Markus E. Nebel, Sebastian Wild, and Conrado Martínez. 2016. Analysis of pivot sampling in dual-pivot quicksort: A holistic analysis of Yaroslavskiy's partitioning scheme. *Algorithmica* 75, 4 (2016), 632–683.

- Naila Rahman. 2002. Algorithms for hardware caches and TLB. In *Algorithms for Memory Hierarchies, Advanced Lectures [Dagstuhl Research Seminar, March 10–14, 2002]*. 171–192.
- Salvador Roura. 2001. Improved master theorems for divide-and-conquer recurrences. *J. ACM* 48, 2 (2001), 170–205. DOI:<http://dx.doi.org/10.1145/375827.375837>
- Peter Sanders and Sebastian Winkel. 2004. Super scalar sample sort. In *Proceedings of the 12th Annual European Symposium on Algorithms (ESA'04)*. Springer, 784–796.
- Robert Sedgewick. 1975. *Quicksort*. Ph.D. Dissertation. Stanford University.
- Robert Sedgewick. 1977. Quicksort with equal keys. *SIAM J. Comput.* 6, 2 (1977), 240–268.
- Kok-Hoi Tan. 1993. *An Asymptotic Analysis of the Number of Comparisons in Multipartition Quicksort*. Ph.D. Dissertation. Carnegie Mellon University.
- M. H. van Emden. 1970. Increasing the efficiency of quicksort. *Commun. ACM* 13, 9 (Sept. 1970), 563–567.
- Sebastian Wild and Markus E. Nebel. 2012. Average case analysis of Java 7's dual pivot quicksort. In *Proceedings of the 20th European Symposium on Algorithms (ESA'12)*. 825–836.
- Sebastian Wild, Markus E. Nebel, and Ralph Neininger. 2015. Average case and distributional analysis of dual-pivot quicksort. *ACM Trans. Algor.* 11, 3 (2015), 22.
- Vladimir Yaroslavskiy. 2009. Replacement of Quicksort in java.util.Arrays with new Dual-Pivot Quicksort (Thread). Retrieved from <http://osdir.com/ml/java-openjdk-core-libs-devel/2009-09/msg00160.html>.

Received October 2015; revised May 2016; accepted June 2016